Unidad Profesional Interdisciplinaria de Ingeniería y Ciencias Sociales y Administrativas (UPIICSA)

Abstracción y uso de Datos

Profesor. Yventz Entzana Garduño

Secuencia: 3CM32

Trabajo: Reporte 2

Integrantes:

García Pamatz Gabriel

Guerrero Hernández Eduardo

Muñoz Sánchez Emilio

Olivera Chávez Valeria

Índice

Contenido

Promedio con puntero al arreglo 3

Nuevo tipo de dato con puntero al arreglo 5

Ordenamiento (burbuja) 8

Ordenamiento (merge sort) 11

Ordenamiento (quick sort) 14

PILA (manera estática por medio de un arreglo) 17

- Dato DB 17
- Nuevo tipo de dato 19

COLA (manera estática por medio de un arreglo) 22

- Dato DB 22
- Nuevo Tipo de dato 24

LISTA (manera estática por medio de un arreglo) 27

- Dato DB 27
- Nuevo tipo de dato 30

PILA (manera dinámica por medio de punteros al mismo tipo de dato) 33

- Dato DB 33
- Nuevo tipo de dato 36

COLA (manera dinámica por medio de punteros al mismo tipo de dato) 39

- Dato DB 39
- Nuevo Tipo de dato 42

LISTA (manera dinámica por medio de punteros al mismo tipo de dato) 45

- Dato DB 45
- Tipo de dato 47

PILA (manera dinámica por medio de las librerías o clases proporcionadas por el IDE) 50

- Nuevo tipo de dato 52

COLA (manera dinámica por medio de las librerías o clases proporcionadas por el IDE) 55

- Dato DB 55
- Nuevo tipo de dato 57

LISTA (manera dinámica por medio de las librerías o clases proporcionadas por el IDE) 60

- Nuevo tipo de dato 63

Promedio con puntero al arreglo

main.cpp Promedio.h Promedio.cpp

```cpp
#include "Promedio.h"
#include<iostream>
using namespace std;

Promedio::Promedio()
{

}


void Promedio::ingresar(){
    float *p = numeros;
    for(int i=0;i<5;i++){
        cout<<"Ingresa numero "<<i+1<<": ";
        cin>>*(p+i);
}
}

void Promedio::calcular(){
    float *p=numeros;
    float promedio=0, maximo=*p, minimo=*p, suma=0;


    for(int i=0;i<5;i++){

        suma+=*(p+i);

        if(*(p+i)>maximo){
            maximo=*(p+i);
        }
        if(*(p+i)<minimo){
            minimo=*(p+i);
        }
    }
    promedio=suma/5;

```

Objetivo del Código

El objetivo de este programa es implementar una clase llamada Promedio para gestionar un conjunto de 5 números reales. A través del uso de **punteros** y estructuras de control, el código busca:

- Solicitar y almacenar datos de entrada del usuario.
- Calcular de manera eficiente el **promedio**.
- Identificar los valores **máximo** y **mínimo** dentro del conjunto de datos.

Explicación del Código

Método de Entrada (ingresar)

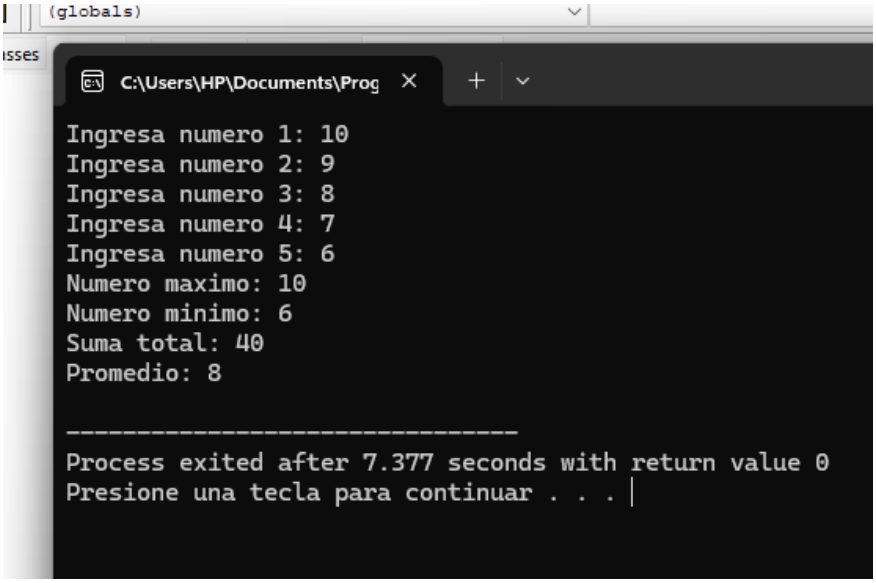
Este método se encarga de llenar el arreglo de números:

- **Uso de Punteros:** Se crea un puntero float *p que apunta a la dirección de memoria del arreglo numeros.
- **Ciclo de Captura:** Utiliza un bucle for del 0 al 4.
- **Aritmética de Punteros:** La instrucción cin >> *(p + i) accede a cada posición de memoria de forma secuencial para guardar el valor ingresado por el usuario.

Método de Procesamiento (calcular)

Es el núcleo lógico del programa donde se realizan los cálculos estadísticos:

1. **Inicialización:** Se definen variables para la suma, el promedio, el maximo y el minimo. Inicializa el máximo y mínimo con el primer valor del arreglo (*p).
2. **Iteración y Comparación:** Mediante un ciclo for, recorre nuevamente el arreglo usando el puntero:
    - **Acumulación:** Suma cada valor a la variable suma.
    - **Lógica de Extremos:** Utiliza condicionales if para comparar el valor actual *(p + i) con los valores guardados en maximo y minimo, actualizándolos si encuentra un número mayor o menor, respectivamente.
3. **Resultado Final:** Al terminar el ciclo, divide la suma total entre 5 para obtener el promedio final.



Que es lo que imprime y explicación:

La salida final después de ingresar los datos es la siguiente:

- Número máximo: 10

- Número mínimo: 6
- Suma total: 40
- Promedio: 8

Explicación: El usuario ingresó una serie descendente: **10, 9, 8, 7 y 6**. El programa almacenó estos valores en un arreglo usando un puntero.

Al comparar cada número, el código determinó que **10** es el valor más alto (Máximo) y **6** es el más bajo (Mínimo).

El programa realizó la operación 10 + 9 + 8 + 7 + 6 = 40 luego Tomó la suma total (**40**) y la dividió entre la cantidad de elementos (**5**), resultando en 40 / 5 = **8** lo que dio como resultado que el promedio final es **8.**

Errores y Soluciones: no hubo errores.

Nuevo tipo de dato con puntero al arreglo

```cpp
#include <iostream>
#include "Auto.h"

using namespace std;

int main(){

    Auto lista[3];
    Auto *p;

    p = lista;

    float precio;
    int anio;
    string marca;

    string nombre, ap, am, genero;
    int edad;

    for(int i = 0; i < 3; i++){

        cout << "\nPersona " << i+1 << endl;

        cout << "Nombre: ";
        cin >> nombre;

        cout << "Apellido paterno: ";
        cin >> ap;

        cout << "Apellido materno: ";
        cin >> am;

        cout << "Genero: ";
        cin >> genero;

        cout << "Edad: ";
        cin >> edad;

        cout << "Marca del auto: ";
        cin >> marca;

        cout << "Precio: ";
        cin >> precio;

        cout << "Anio: ";
        cin >> anio;

        (p + i)->setDatos(precio, anio, marca, nombre, ap, am, genero, edad);
    }

    cout << "\n----- LISTA DE PERSONAS Y AUTOS -----\n";

    for(int i = 0; i < 3; i++){
        (p + i)->mostrarDatos();
    }

    return 0;
}
```

Objetivo del Código

El objetivo de este programa es gestionar un sistema de registro para 3 personas y sus respectivos vehículos utilizando **arreglos de objetos** y **punteros**. El código busca:

- Capturar datos personales (nombre, apellidos, género, edad) y datos del vehículo (marca, precio, año).

- Utilizar la **aritmética de punteros** para acceder y manipular objetos dentro de un arreglo.
- Implementar el concepto de **encapsulamiento** mediante el uso de métodos set para asignar valores y métodos de visualización para mostrar la información almacenada.

Explicación del Código

Se crea un arreglo de objetos de tipo Auto llamado lista con capacidad para 3 elementos.

- Se declara un puntero Auto *p y se vincula al inicio del arreglo (p = lista). Esto permite recorrer los objetos sin usar los índices tradicionales [].

Ciclo de Captura de Datos

Se utiliza un bucle for que se repite 3 veces para solicitar la información de cada persona y su auto:

- **Variables Temporales:** Se usan variables como nombre, ap, am, marca, precio, etc., para almacenar lo que el usuario escribe.
- **Asignación con Punteros:** La línea clave es (p + i)->setDatos(...). Aquí, el puntero se desplaza i posiciones en la memoria para alcanzar al objeto correspondiente y llama al método setDatos para guardar toda la información capturada en los atributos privados de dicho objeto.

Una vez que todos los datos han sido guardados, el programa imprime una cabecera y utiliza un segundo bucle for:

- Nuevamente, usa la sintaxis de punteros (p + i)->mostrarDatos() para invocar la función que imprime en pantalla el resumen de la información de cada persona y su auto.

```
Persona 1
Nombre: Emilio
Apellido paterno: Muñoz
Apellido materno: Sanchez
Genero: Masculino
Edad: 21
Marca del auto: Bentley
Precio: 10000
Anio: 2022

Persona 2
Nombre: Eduardo
Apellido paterno: Guerrero
Apellido materno: Hernandez
Genero: Masculino
Edad: 19
Marca del auto: Chevrolet
Precio: 12000
Anio: 2010

Persona 3
Nombre: Vaaleria
Apellido paterno: Olivera
Apellido materno: Chavez
Genero: Femenino
Edad: 20
Marca del auto: Bw
Precio: 20000
Anio: 2018
```

```
----- LISTA DE PERSONAS Y AUTOS -----

DATOS DEL PROPIETARIO
Nombre: Emilio Muñoz Sanchez
Genero: Masculino
Edad: 21

DATOS DEL AUTO
Marca: Bentley
Precio: 10000
Anio: 2022

DATOS DEL PROPIETARIO
Nombre: Eduardo Guerrero Hernandez
Genero: Masculino
Edad: 19

DATOS DEL AUTO
Marca: Chevrolet
Precio: 12000
Anio: 2010

DATOS DEL PROPIETARIO
Nombre: Vaaleria Olivera Chavez
Genero: Femenino
Edad: 20
```

Que es lo que imprime y explicación:

El programa imprime dos secciones principales: la captura de datos y el reporte final organizado.

1. Entrada de Datos (Fase de captura)

Primero, el programa solicita la información para tres registros:

Persona 1: Emilio Muñoz Sanchez, Masculino, 21 años. Auto: Bentley, 2022, $10,000.

Persona 2: Eduardo Guerrero Hernandez, Masculino, 19 años. Auto: Chevrolet, 2010, $12,000.

Persona 3: Valeria Olivera Chavez, Femenino, 20 años. Auto: Bw, 2018, $20,000.

Salida de Información (Lista de Personas y Autos)

Una vez terminada la captura, el programa imprime el encabezado

LISTA DE PERSONAS Y AUTOS y despliega los datos de forma estructurada para cada entrada:

- **DATOS DEL PROPIETARIO:** Muestra el nombre completo (concatenando nombre y apellidos), género y edad.
- **DATOS DEL AUTO:** Muestra la marca, el precio y el año del vehículo asociado.

Errores y Soluciones: no hubo errores.

Ordenamiento (burbuja)

main.cpp | Persona.h | Clase_abstracta.h | Clase_concreta_Burbuja.h

```cpp
#include <iostream>
#include <vector>
#include "Clase_concreta_Burbuja.h"
#include <limits>
using namespace std;

void mostrar(vector<Persona>& personas) {
    for (int i = 0; i < personas.size(); i++) {
    personas[i].mostrar();
}
}

int main() {
    vector<Persona> personas;
    Clase_concreta_Burbuja ordenador;
    int op;

    do {
        cout << "\n1. Agregar\n2. Mostrar\n3. Ordenar Burbuja\n4. Salir\nOpcion: ";
        cin >> op;

        if (op == 1) {
            string nombre;
            int edad;
            cout << "Nombre: ";
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            getline(cin, nombre);
            cout << "Edad: ";
            cin >> edad;

            if (cin.fail()) {
                cin.clear();
                cin.ignore(1000, '\n');
                cout << "Entrada invalida\n";
                continue;
                }
```

```cpp
                personas.push_back(Persona(nombre, edad));
            }
            else if (op == 2) {
                mostrar(personas);
            }
            else if (op == 3) {
                ordenador.ordenar(personas);
                cout << "Ordenado.\n";
            }

        } while (op != 4);

        return 0;
    }
```

Objetivo del Código

El objetivo de este programa es implementar un sistema de gestión de registros de personas que permite el almacenamiento dinámico y el ordenamiento de datos. El código busca:

- Utilizar un contenedor **std::vector** para almacenar objetos de la clase Persona de forma dinámica (sin un límite fijo como los arreglos tradicionales).
- Implementar una arquitectura basada en **clases abstractas y concretas** para realizar tareas de ordenamiento.
- Gestionar un menú interactivo con validación de entrada de datos

Explicación del Código

Es una función auxiliar que recibe el vector de personas por referencia (&) para ahorrar memoria. Recorre cada elemento del vector y llama al método .mostrar() propio de cada objeto Persona para imprimir sus datos.

- **Contenedor Dinámico:** Se declara vector<Persona> personas, que crece automáticamente conforme agregamos elementos.
- **Clases de Ordenamiento:** Se instancia Clase_concreta_Burbuja ordenador, lo que sugiere que el programa utiliza el algoritmo de **burbuja** para organizar a las personas (posiblemente por edad o nombre).
- **Menú Interactivo (do-while):**
  - **Opción 1 (Agregar):** Solicita nombre y edad. Es importante notar el uso de cin.ignore() y getline(), que sirven para capturar nombres completos con espacios correctamente. También incluye una validación con cin.fail() por si el usuario ingresa letras en lugar de números en la edad.
  - **Opción 2 (Mostrar):** Llama a la función global para listar a todos los registrados.
  - **Opción 3 (Ordenar):** Utiliza el objeto ordenador para ejecutar el algoritmo sobre el vector de personas.
  - **Opción 4 (Salir):** Finaliza el ciclo y el programa.

```
1. Agregar
2. Mostrar
3. Ordenar Burbuja
4. Salir
Opcion: 2
Nombre: Valeria | Edad: 19
Nombre: lalo | Edad: 21
Nombre: Emilio | Edad: 15

1. Agregar
2. Mostrar
3. Ordenar Burbuja
4. Salir
Opcion: 3
Ordenado.

1. Agregar
2. Mostrar
3. Ordenar Burbuja
4. Salir
Opcion: 2
Nombre: Emilio | Edad: 15
Nombre: Valeria | Edad: 19
Nombre: lalo | Edad: 21

1. Agregar
2. Mostrar
3. Ordenar Burbuja
4. Salir
Opcion: |
```

Que es lo que imprime y explicación:

La consola te da para elegir 4 opciones

1. Agregar
2. Mostrar
3. Ordenar Burbuja
4. Salir

Al seleccionar la opción 1 se van agregando el nombre de las personas con sus edades después al seleccionar la opción 2, el programa imprime la lista de personas en el orden en que fueron agregadas:

- Valeria | Edad: 19
- lalo | Edad: 21
- Emilio | Edad: 15

Al elegir la opción 3, el programa imprime el mensaje "Ordenado.", confirmando que el algoritmo ha procesado el vector de personas.

Resultado Final: seleccionar nuevamente la opción 2, se imprime la lista reorganizada de menor a mayor según la edad:

- Emilio | Edad: 15
- Valeria | Edad: 19

- lalo | Edad: 21

Errores y Soluciones: no hubo errores.

Ordenamiento (merge sort)

main.cpp | Clase_abstracta.h | Clase_concreta_Merge.h | Persona.h

```cpp
#include <iostream>
#include <vector>
#include <limits>
#include "Clase_concreta_Merge.h"
using namespace std;

void mostrar(vector<Persona>& personas) {
    for (int i = 0; i < personas.size(); i++) {
    personas[i].mostrar();
}
}

int main() {
    vector<Persona> personas;
    Clase_concreta_Merge ordenador;
    int op;

    do {
        cout << "\n1. Agregar\n2. Mostrar\n3. Ordenar MergeSort\n4. Salir\nOpcion: ";
        cin >> op;

        if (op == 1) {
            string nombre;
            int edad;
            cout << "Nombre: ";
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            getline(cin, nombre);
            cout << "Edad: ";
            cin >> edad;
            if (cin.fail()) {
                cin.clear();
                cin.ignore(1000, '\n');
                cout << "Entrada invalida\n";
                continue;
}
            personas.push_back(Persona(nombre, edad));
        }
        else if (op == 2) {
            mostrar(personas);
        }
        else if (op == 3) {
            ordenador.ordenar(personas);
            cout << "Ordenado.\n";
        }

    } while (op != 4);

    return 0;
}
```

Objetivo del Código

El objetivo de este programa es gestionar un registro dinámico de personas y aplicar un algoritmo de ordenamiento avanzado llamado **MergeSort** (Ordenamiento por Mezcla). El código busca:

- Almacenar objetos de tipo Persona en un contenedor dinámico std::vector.
- Implementar un método de ordenamiento basado en la técnica de **"Divide y Vencerás"** para mejorar la eficiencia respecto a métodos más sencillos.
- Mantener una arquitectura modular mediante el uso de una clase específica para el ordenamiento (Clase_concreta_Merge).

Explicación del Código

A diferencia del de"Burbuja", este programa instancia Clase_concreta_Merge ordenador.

- **MergeSort** es un algoritmo mucho más eficiente para grandes cantidades de datos.
- Funciona dividiendo el vector a la mitad repetidamente hasta tener elementos individuales y luego los mezcla (*merges*) de forma ordenada.

Cuando seleccionas la **Opción 1**, el programa crea un nuevo objeto Persona con los datos capturados y lo inserta al final del vector usando personas.push_back().

Este método es eficiente porque el vector gestiona automáticamente el crecimiento de la memoria necesaria.

- Al elegir la **Opción 3**, se invoca ordenador.ordenar(personas). Internamente, este método realiza la recursión necesaria para reorganizar los objetos Persona dentro del vector, usualmente tomando como referencia un atributo numérico (como la edad).

```
Nombre: Fernanda
Edad: 10

1. Agregar
2. Mostrar
3. Ordenar MergeSort
4. Salir
Opcion: 1
Nombre: Natalia 8
Edad: 8

1. Agregar
2. Mostrar
3. Ordenar MergeSort
4. Salir
Opcion: 2
Nombre: Andrea | Edad: 18
Nombre: Fernanda | Edad: 10
Nombre: Natalia 8 | Edad: 8

1. Agregar
2. Mostrar
3. Ordenar MergeSort
4. Salir
Opcion: 3
Ordenado.

1. Agregar
2. Mostrar
3. Ordenar MergeSort
4. Salir
Opcion: 2
Nombre: Natalia 8 | Edad: 8
Nombre: Fernanda | Edad: 10
Nombre: Andrea | Edad: 18
```

Que es lo que imprime y explicación:

La consola muestra el flujo completo de gestión de datos:

- Fase de Agregado (Opción 1): Se añade a una nueva persona
- Estado Inicial (Opción 2): Al mostrar la lista por primera vez, las personas aparecen en el orden en que estaban almacenadas en el vector:
    - Andrea | Edad: 18
    - Fernanda | Edad: 10
    - Natalia 8 | Edad: 8
- Acción de Ordenamiento (Opción 3): Se ejecuta el algoritmo MergeSort y el programa imprime la confirmación: "Ordenado.".

- **Resultado Final (Opción 2):** Al mostrar la lista nuevamente, el programa imprime los registros ordenados **de menor a mayor según la edad**:

- Natalia 8 | Edad: 8
- Fernanda | Edad: 10
- Andrea | Edad: 18

Errores y Soluciones: no hubo errores.

Ordenamiento (quick sort)

main.cpp | Clase_abstracta.h | Clase_concreta_Quick.h | Persona.h

```cpp
#include <iostream>
#include <vector>
#include "Clase_concreta_Quick.h"
#include <limits>

using namespace std;

void mostrar(vector<Persona>& personas) {
    for (int i = 0; i < personas.size(); i++) {
    personas[i].mostrar();
}
}

int main() {
    vector<Persona> personas;
    Clase_concreta_Quick ordenador;
    int op;

    do {
        cout << "\n1. Agregar\n2. Mostrar\n3. Ordenar QuickSort\n4. Salir\nOpcion: ";
        cin >> op;

        if (op == 1) {
            string nombre;
            int edad;
            cout << "Nombre: ";
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            getline(cin, nombre);
            cout << "Edad: ";
            cin >> edad;
            if (cin.fail()) {
                cin.clear();
                cin.ignore(1000, '\n');
                cout << "Entrada invalida\n";
                continue;
}
```

```cpp
}
            personas.push_back(Persona(nombre, edad));
        }
        else if (op == 2) {
            mostrar(personas);
        }
        else if (op == 3) {
            ordenador.ordenar(personas);
            cout << "Ordenado.\n";
        }

    } while (op != 4);

    return 0;
}
```

Objetivo del Código

El objetivo de este programa es gestionar un sistema de registros dinámicos de personas y aplicar el algoritmo de ordenamiento **QuickSort** (Ordenamiento Rápido). El código busca:

- Utilizar la estructura dinámica std::vector para el almacenamiento flexible de objetos Persona.
- Implementar un método de ordenamiento basado en la técnica de **particionamiento**, reconocido por su alta eficiencia en el manejo de grandes volúmenes de datos.
- Mantener la modularidad del sistema mediante la especialización de la clase Clase_concreta_Quick.

Explicación del Código

Se instancia el objeto Clase_concreta_Quick ordenador.

- **QuickSort** funciona seleccionando un elemento llamado **"pivote"** y reorganizando el vector de modo que los elementos menores al pivote queden a la izquierda y los mayores a la derecha.
- Es un algoritmo de complejidad promedio, lo que lo hace extremadamente eficiente.

Validación

- El código utiliza cin.ignore y numeric_limits para limpiar el búfer de entrada. Esto es esencial cuando se mezcla la lectura de números (cin >> op) con cadenas de texto (getline), evitando que el programa se salte la captura del nombre.
- Incluye un bloque de control if (cin.fail()) que detecta si el usuario ingresó un dato no numérico en la edad, limpiando el error y permitiendo que el programa continúe sin cerrarse.

Inserción Dinámica

- Usa personas.push_back(Persona(nombre, edad)). Esta instrucción crea un objeto temporal de la clase Persona y lo "empuja" al final del vector, gestionando automáticamente la memoria necesaria.

```
1. Agregar
2. Mostrar
3. Ordenar QuickSort
4. Salir
Opcion: 1
Nombre: Gabriel
Edad: 25

1. Agregar
2. Mostrar
3. Ordenar QuickSort
4. Salir
Opcion: 1
Nombre: Valeria
Edad: 15

1. Agregar
2. Mostrar
3. Ordenar QuickSort
4. Salir
Opcion: 2
Nombre: Gabriel | Edad: 25
Nombre: Valeria | Edad: 15
```

```
1. Agregar
2. Mostrar
3. Ordenar QuickSort
4. Salir
Opcion: 3
Ordenado.

1. Agregar
2. Mostrar
3. Ordenar QuickSort
4. Salir
Opcion: 2
Nombre: Valeria | Edad: 15
Nombre: Gabriel | Edad: 25
```

Que es lo que imprime y explicación:

La consola muestra el ciclo de vida de los datos mediante el algoritmo QuickSort:

1. Captura de Datos (Opción 1): El usuario agrega a dos personas:
    - Gabriel, con 25 años.
    - Valeria, con 15 años.
2. (Opción 2): Al mostrar los datos inicialmente, aparecen en el orden en que fueron ingresados:
    - Nombre: Gabriel | Edad: 25.
    - Nombre: Valeria | Edad: 15.
3. Proceso de Ordenamiento (Opción 3): Se ejecuta la lógica de la clase Clase_concreta_Quick y el programa confirma la acción con el mensaje: "Ordenado.".
4. Estado Ordenado (Opción 2): Tras el ordenamiento, los registros aparecen reorganizados de menor a mayor edad:
    - Nombre: Valeria | Edad: 15.
    - Nombre: Gabriel | Edad: 25.

Errores y Soluciones: no hubo errores.

PILA (manera estática por medio de un arreglo)

Dato DB

main.cpp Estructura.h Pila.h

```cpp
#ifndef PILA_H
#define PILA_H

#include <iostream>
#include "Estructura.h"

using namespace std;

class Pila : public Estructura {
private:
    int* arreglo;
    int tope;
    int capacidad;

public:
    Pila(int cap) {
        capacidad = cap;
        arreglo = new int[cap];
        tope = -1;
    }

    void agregar() { // push
        if (!estaLlena()) {
            int valor;
            cout << "Ingresa un numero: ";
            cin >> valor;

            tope++;
            arreglo[tope] = valor;

            cout << "Elemento agregado.\n";
        } else {
            cout << "La pila esta llena.\n";
        }
    }
```

main.cpp Estructura.h Pila.h

```cpp
    void quitar() { // pop
        if (!estaVacia()) {
            cout << "Elemento eliminado: " << arreglo[tope] << endl;
            tope--;
        } else {
            cout << "La pila esta vacia.\n";
        }
    }

    void mostrar() {
        if (estaVacia()) {
            cout << "La pila esta vacia.\n";
            return;
        }

        cout << "Contenido (tope a base): ";
        for (int i = tope; i >= 0; i--) {
            cout << arreglo[i] << " ";
        }
        cout << endl;
    }

    bool estaVacia() {
        return tope == -1;
    }

    bool estaLlena() {
        return tope == capacidad - 1;
    }

    int tamanio() {
        return tope + 1;
    }
};

#endif
```

Objetivo del Código

El propósito de este archivo es definir una estructura de datos tipo **LIFO** (*Last In, First Out* - Último en entrar, Primero en salir). Permite almacenar números enteros de forma organizada, controlando el acceso únicamente a través del "tope" de la estructura.

Explicación de los Componentes (Atributos)

En la sección private encontramos el corazón de la pila:

- **int* arreglo:** Un puntero que se utiliza para crear un arreglo dinámico donde se guardarán los números.
- **int tope:** Un índice que indica la posición del último elemento agregado. Comienza en -1 porque la pila está vacía al inicio.
- **int capacidad:** Define el tamaño máximo que tendrá la pila para evitar que se desborde.

Agregar (Push): Es la operación para insertar datos.

1. Verifica si hay espacio con !estaLlena().
2. Incrementa el tope.

3. Guarda el valor en esa posición.
4. Si no hay espacio, imprime: "La pila esta llena.".

Quitar (Pop)

Es la operación para extraer datos.

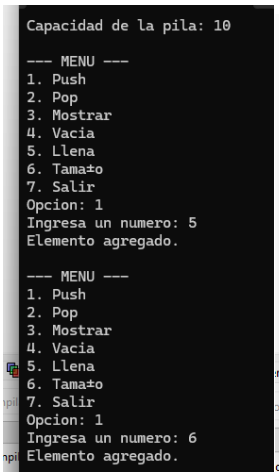
1. Solo funciona si !estaVacia().
2. Muestra el elemento que está en el tope y luego reduce el índice tope--. El elemento no se borra físicamente de inmediato, pero el programa "olvida" su posición, haciéndolo inaccesible.

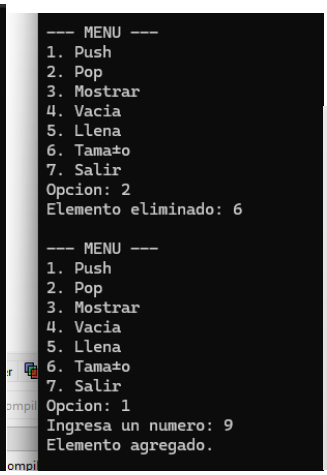
Mostrar

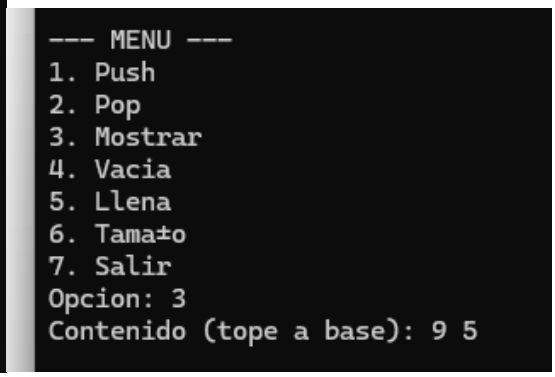
Recorre el arreglo desde el tope hasta el índice 0. Esto es importante porque las pilas se leen de arriba hacia abajo (del más reciente al más antiguo).

Funciones de Control

- **estaVacia():** Devuelve true si el tope es -1.
- **estaLlena():** Devuelve true si el tope ha alcanzado el límite de la capacidad (capacidad - 1).
- **tamanio():** Indica cuántos elementos hay actualmente (si el tope es 0, hay 1 elemento).







Que es lo que imprime y explicación:

El programa imprime el rastro de una serie de operaciones sobre una pila con capacidad para 10 elementos:

1. Inserciones Iniciales (Push):
    - Se agrega el número 5. El programa imprime: "Elemento agregado.".

- Se agrega el número 6. El programa imprime: "Elemento agregado.". En este punto, el 6 está en el tope de la pila.

2. Eliminación (Pop):
    - Se selecciona la opción 2. Como el 6 fue el último en entrar, es el primero en salir. El programa imprime: "Elemento eliminado: 6".
3. Nueva Inserción:
    - Se agrega el número 9. El programa imprime: "Elemento agregado.". Ahora el tope es 9 y debajo está el 5.
4. Visualización (Mostrar):
    - Al elegir la opción 3, el programa recorre el arreglo desde el tope hasta la base. Imprime: "Contenido (tope a base): 9 5".

Errores y Soluciones: no hubo errores.

Nuevo tipo de dato

main.cpp Estructura.h Pila.h

```cpp
#ifndef PILA_H
#define PILA_H

#include <iostream>
#include <string>
#include "Estructura.h"

using namespace std;

class Persona {
public:
    string nombre;
    int edad;

    Persona() {
        nombre = "";
        edad = 0;
    }

    Persona(string n, int e) {
        nombre = n;
        edad = e;
    }
};

class Pila : public Estructura {
private:
    Persona* arreglo;
    int tope;
    int capacidad;

public:
    Pila(int cap) {
        capacidad = cap;
        arreglo = new Persona[cap];
        tope = -1;
    }

    void agregar() {
        if (!estaLlena()) {
            string nombre;
            int edad;

            cout << "Nombre: ";
            cin >> nombre;
            cout << "Edad: ";
            cin >> edad;

            tope++;
            arreglo[tope] = Persona(nombre, edad);

            cout << "Persona agregada.\n";
        } else {
            cout << "La pila esta llena.\n";
        }
    }

    void quitar() {
        if (!estaVacia()) {
            cout << "Eliminando: "
                 << arreglo[tope].nombre
                 << " (" << arreglo[tope].edad << ")\n";

            tope--;
        } else {
            cout << "La pila esta vacia.\n";
        }
    }

    void mostrar() {
        if (estaVacia()) {
            cout << "La pila esta vacia.\n";
            return;
        }

        cout << "Contenido (tope a base):\n";
        for (int i = tope; i >= 0; i--) {
            cout << arreglo[i].nombre
                 << " - " << arreglo[i].edad << endl;
        }
    }

    bool estaVacia() {
        return tope == -1;
    }

    bool estaLlena() {
        return tope == capacidad - 1;
    }

    int tamanio() {
        return tope + 1;
    }
};

#endif
```

Objetivo del Código

El objetivo de este programa es implementar una estructura de datos tipo **Pila (Stack)** utilizando **memoria dinámica** para almacenar y gestionar objetos complejos de la clase Persona. Busca demostrar cómo una estructura **LIFO** (*Last In, First Out*) puede organizar información del mundo real (nombres y edades), permitiendo operaciones de inserción, extracción y visualización controladas por un índice de tope.

Explicación del código

La Entidad de Datos (Clase Persona)

Antes de la pila, el código define la clase Persona, que actúa como el molde para los datos:

- **Atributos:** Contiene un string nombre y un int edad.
- **Constructores:** Incluye un constructor por defecto para inicializar valores y uno con parámetros para facilitar la creación de objetos al momento de hacer el *push*.

Gestión de Memoria Dinámica

En la clase Pila, el atributo Persona* arreglo es un puntero que reserva un bloque de memoria específico para objetos de tipo Persona. Al usar new Persona[cap] en el constructor, el programa asegura que haya espacio suficiente para la cantidad de registros definida por el usuario.

Operaciones Fundamentales

- **Agregar (Push):** El método agregar() (Línea 39) solicita los datos al usuario, incrementa el tope y asigna un nuevo objeto Persona a esa posición del arreglo dinámico.
- **Quitar (Pop):** El método quitar() (Línea 58) accede a los atributos .nombre y .edad de la persona en el tope para mostrarlos antes de reducir el índice tope--, lo que efectivamente "elimina" el acceso a ese registro.
- **Mostrar:** La función mostrar() (Línea 70) recorre la pila de forma inversa (desde tope hasta 0), lo cual es esencial para respetar la lógica de una pila donde lo último que entró es lo primero que se visualiza.

Funciones de Estado

El programa incluye métodos de validación críticos:

- **estaVacia():** Retorna true si tope == -1, evitando errores al intentar quitar elementos de una pila sin datos.
- **estaLlena():** Retorna true si el tope alcanzó capacidad - 1, previniendo el desbordamiento de memoria (Stack Overflow).

```
Capacidad de la pila: 1

--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 1
Nombre: Valeria
Edad: 12
Persona agregada.

--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 1
La pila esta llena.
```

Que es lo que imprime y explicación:

1. **Configuración Inicial:** Se establece una **Capacidad de la pila: 1**. Esto significa que el arreglo dinámico solo tiene espacio para un objeto de tipo Persona.
2. **Primera Inserción (Éxito):**
    - Se selecciona la **Opcion: 1** (Push).
    - Se ingresan los datos: Nombre: **Valeria**, Edad: **12**.
    - El programa imprime: "Persona agregada.".
3. **Segunda Inserción (Fallo):**
    - Se intenta de nuevo la **Opcion: 1**.
    - Como la capacidad es 1 y ya hay un elemento, el programa imprime inmediatamente: "La pila esta llena.".

Errores y Soluciones: no hubo errores.

COLA (manera estática por medio de un arreglo)

Dato DB

main.cpp Estructura.h Cola.h

```cpp
#ifndef COLA_H
#define COLA_H

#include <iostream>
#include "Estructura.h"

using namespace std;

class Cola : public Estructura {
private:
    int* arreglo;
    int frente;
    int fin;
    int capacidad;

public:
    Cola(int cap) {
        capacidad = cap;
        arreglo = new int[cap];
        frente = 0;
        fin = -1;
    }

    void agregar() override {
        if (!estaLlena()) {
            int valor;
            cout << "Ingresa un numero: ";
            cin >> valor;

            fin++;
            arreglo[fin] = valor;

            cout << "Elemento agregado.\n";
        } else {
            cout << "La cola esta llena.\n";
        }
```

```cpp
    }

    void quitar() override {
        if (!estaVacia()) {
            cout << "Elemento eliminado: " << arreglo[frente] << endl;
            frente++;
        } else {
            cout << "La cola esta vacia.\n";
        }
    }

    void mostrar() override {
        if (estaVacia()) {
            cout << "La cola esta vacia.\n";
            return;
        }

        cout << "Contenido: ";
        for (int i = frente; i <= fin; i++) {
            cout << arreglo[i] << " ";
        }
        cout << endl;
    }

    bool estaVacia() override {
        return frente > fin;
    }

    bool estaLlena() override {
        return fin == capacidad - 1;
    }

    int tamanio() override {
        return (fin - frente + 1);
    }
```

Objetivo del Código

El objetivo de este programa es implementar una estructura de datos lineal de tipo **FIFO** (*First In, First Out* - Primero en entrar, Primero en salir) utilizando memoria dinámica. Busca gestionar una colección de elementos donde las inserciones se realizan por un extremo (final) y las extracciones por el otro (frente), garantizando el orden cronológico de los datos.

Explicación del código

Atributos de Control

A diferencia de la pila que solo usa un "tope", la cola necesita dos punteros o índices para funcionar correctamente:

- **int frente:** Indica la posición del primer elemento de la fila (el que sigue para salir).
- **int fin:** Indica la posición del último elemento agregado.
- **int* arreglo:** El espacio de memoria dinámica donde se guardan los enteros.

Operación de Inserción: agregar()

Conocida como **Enqueue**:

1. Verifica si hay espacio libre al final del arreglo con !estaLlena().
2. Incrementa el índice fin++.

3. Coloca el nuevo valor en esa posición.
4. Si ya no hay espacio según la capacidad inicial, bloquea la entrada.

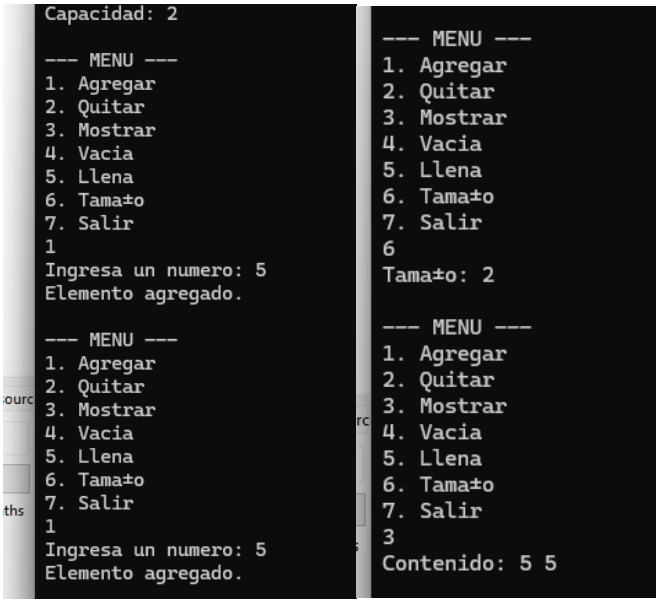
Operación de Extracción: quitar()

Conocida como **Dequeue**:

1. Verifica que la cola no esté vacía.
2. Muestra el elemento que está en el arreglo[frente].
3. **La clave:** Incrementa frente++. Al hacer esto, el "primer lugar" se recorre al siguiente elemento, dejando el anterior fuera de la estructura lógica.

Visualización y Control

- **mostrar():** Recorre el arreglo desde el índice frente hasta el fin. Esto asegura que siempre veamos los elementos en el orden exacto en que van a ser atendidos.
- **estaVacia():** La cola se considera vacía cuando el frente supera al fin (frente > fin), lo que significa que ya sacamos todos los elementos que habíamos metido.
- **tamanio():** Calcula cuántos elementos quedan restando los índices: (fin - frente + 1).



Que es lo que imprime y explicación:

Configuración de Memoria: Se establece una Capacidad: 2.

Llenado de la Estructura:

- Se selecciona la opción 1 y se ingresa el número 5. El programa imprime: "Elemento agregado.".
- Se selecciona nuevamente la opción 1 y se ingresa otro número 5. El programa imprime: "Elemento agregado.".

Consulta de Tamaño (Opción 6):

- El usuario solicita ver el tamaño actual. El programa imprime: Tamaño: 2.

Visualización de Contenido (Opción 3):

- El usuario solicita mostrar los datos. El programa imprime: Contenido: 5 5.

Errores y Soluciones: no hubo errores.

Nuevo Tipo de dato

main.cpp Estructura.h Cola.h

```cpp
#ifndef COLA_H
#define COLA_H

#include <iostream>
#include <string>
#include "Estructura.h"

using namespace std;

class Persona {
public:
    string nombre;
    int edad;

    Persona() {}

    Persona(string n, int e) {
        nombre = n;
        edad = e;
    }
};

class Cola : public Estructura {
private:
    Persona* arreglo;
    int frente;
    int fin;
    int capacidad;

public:
    Cola(int cap) {
        capacidad = cap;
        arreglo = new Persona[cap];
        frente = 0;
        fin = -1;
    }

    void agregar() override {
        if (!estaLlena()) {
            string nombre;
            int edad;

            cout << "Nombre: ";
            cin >> nombre;
            cout << "Edad: ";
            cin >> edad;

            fin++;
            arreglo[fin] = Persona(nombre, edad);

            cout << "Persona agregada.\n";
        } else {
            cout << "La cola esta llena.\n";
        }
    }

    void quitar() override {
        if (!estaVacia()) {
            Persona p = arreglo[frente];
            cout << "Eliminando: " << p.nombre << " (" << p.edad << ")\n";
            frente++;
        } else {
            cout << "La cola esta vacia.\n";
        }
    }

    void mostrar() override {
        if (estaVacia()) {
            cout << "La cola esta vacia.\n";
            return;
        }
        cout << "Contenido:\n";
        for (int i = frente; i <= fin; i++) {
            cout << arreglo[i].nombre << " - " << arreglo[i].edad << endl;
        }
    }

    bool estaVacia() override {
        return frente > fin;
    }

    bool estaLlena() override {
        return fin == capacidad - 1;
    }

    int tamanio() override {
        return (fin - frente + 1);
    }
};

#endif
```

Objetivo del Código

El objetivo principal es implementar una estructura de datos lineal de tipo **FIFO** (*First In, First Out*) que sea capaz de gestionar entidades complejas. A diferencia de una cola de números simples, este programa permite organizar una fila de "Personas", manteniendo sus atributos

individuales (nombre y edad) y respetando el orden de llegada para su procesamiento o eliminación.

Explicación del Código

1. La Clase Persona

Es el contenedor de los datos. Define qué es una "Persona" para el programa:

- **Atributos:** nombre (texto) y edad (entero).
- **Constructor:** Permite crear una persona vacía o una con datos específicos rápidamente mediante Persona(nombre, edad).

2. Estructura de la Cola

La clase Cola hereda de Estructura y utiliza punteros para manejar objetos:

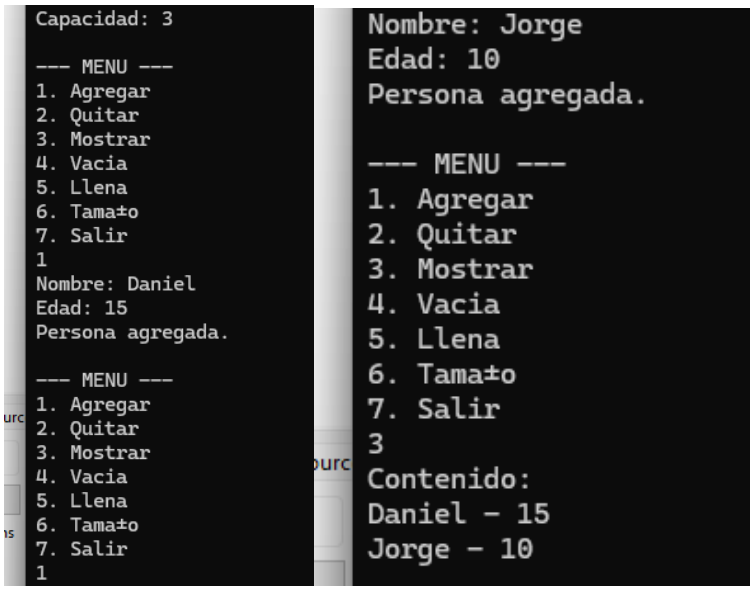
- **Persona* arreglo:** Es un puntero que apunta a un bloque de memoria donde se guardarán múltiples objetos Persona.
- **frente y fin:** Son los controladores de la fila. El frente indica quién sigue para salir y el fin indica dónde se formará el siguiente.

3. Operaciones Principales

- **agregar()**
    1. Verifica si hay espacio (!estaLlena).
    2. Pide el nombre y la edad por teclado.
    3. Incrementa el índice fin.
    4. Crea un nuevo objeto Persona y lo coloca en arreglo[fin].
- **quitar()**
    1. Toma el objeto que está en la posición frente.
    2. Imprime un mensaje indicando quién está siendo "eliminado" de la fila.
    3. Incrementa el frente++. Al hacer esto, esa persona ya no "existe" para la cola, aunque siga en memoria.
- **mostrar()**
    1. Recorre el arreglo desde el índice frente hasta el fin.
    2. Imprime la lista de personas actualmente formadas con el formato: Nombre - Edad.

4. Lógica de Control

- **estaVacia():** Retorna verdadero si frente > fin. Esto sucede cuando todos los que entraron ya salieron.
- **estaLlena():** Retorna verdadero si fin llegó al límite de la capacidad - 1.
- **tamanio():** Calcula cuántas personas hay en la fila restando las posiciones de los índices.



Que es lo que imprime y explicación: lo que imprime es lo siguiente

Configuración: Se define una Capacidad: 3 para la cola.

Primera Inserción:

- Se selecciona la opción 1 (Agregar).
- Se ingresa Nombre: Daniel, Edad: 15.
- El programa imprime: "Persona agregada.".

Segunda Inserción:

- Se selecciona la opción 1 (Agregar).
- Se ingresa Nombre: Jorge, Edad: 10.
- El programa imprime: "Persona agregada.".

Visualización (Opción 3):

- El programa imprime la etiqueta "Contenido:".
- Lista a los integrantes en orden:
    - Daniel – 15

- Jorge – 10

Errores y Soluciones: no hubo errores.

LISTA (manera estática por medio de un arreglo)

Dato DB

main.cpp | Estructura.h | Lista.h

```cpp
#ifndef LISTA_H
#define LISTA_H

#include <iostream>
#include "Estructura.h"

using namespace std;

class Lista : public Estructura {
private:
    int* arreglo;
    int capacidad;
    int elementos;

public:
    Lista(int cap) {
        capacidad = cap;
        elementos = 0;
        arreglo = new int[cap];
    }

    void agregar() {
        if (!estaLlena()) {
            int valor;
            cout << "Ingresa un numero: ";
            cin >> valor;

            arreglo[elementos] = valor;
            elementos++;

            cout << "Elemento agregado.\n";
        } else {
            cout << "La lista esta llena.\n";
        }
    }

    void quitar() {
        if (!estaVacia()) {
            int pos;
            cout << "Posicion a eliminar (0 a " << elementos-1 << "): ";
            cin >> pos;

            if (pos >= 0 && pos < elementos) {
                cout << "Eliminado: " << arreglo[pos] << endl;

                for (int i = pos; i < elementos - 1; i++) {
                    arreglo[i] = arreglo[i + 1];
                }

                elementos--;
            } else {
                cout << "Posicion invalida.\n";
            }
        } else {
            cout << "La lista esta vacia.\n";
        }
    }

    void mostrar() {
        if (estaVacia()) {
            cout << "La lista esta vacia.\n";
            return;
        }

        cout << "Contenido: ";
        for (int i = 0; i < elementos; i++) {
            cout << arreglo[i] << " ";
        }
        cout << endl;
    }

    bool estaVacia() {
        return elementos == 0;
    }

    bool estaLlena() {
        return elementos == capacidad;
    }

    int tamanio() {
        return elementos;
    }
};

#endif
```

Objetivo del Código

El objetivo es implementar una estructura de datos lineal que permita el almacenamiento de elementos en posiciones contiguas de memoria, ofreciendo la flexibilidad de acceder y **eliminar elementos en cualquier posición específica** (acceso aleatorio). A diferencia de las estructuras LIFO o FIFO, esta lista busca gestionar un conjunto de datos donde el orden no está restringido a los extremos.

Explicación del código

Atributos de la Clase

- **int* arreglo:** Puntero para gestionar la memoria dinámica.

- **int capacidad:** Define el tamaño máximo físico de la lista.
- **int elementos:** Actúa como un contador de cuántos datos hay realmente almacenados. También indica la posición donde se insertará el siguiente elemento.

Operación de Inserción: agregar()

Es una inserción al final:

1. Verifica si hay espacio con !estaLlena().
2. Inserta el valor en la posición actual de elementos.
3. Incrementa el contador elementos++.

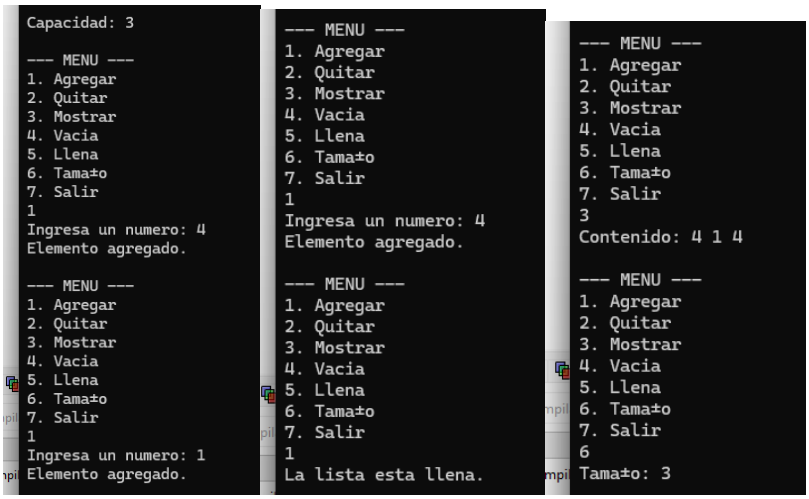
Operación Crítica: quitar()

Esta es la parte más compleja del código porque debe mantener la lista compacta:

1. **Validación:** Pide una posición (pos) y verifica que esté dentro del rango válido (entre 0 y elementos - 1).
2. **Corrimiento (Shifting):** Al eliminar un elemento intermedio, queda un "hueco". El bucle for (Línea 46) recorre los elementos desde la posición eliminada hacia adelante, moviendo cada valor una posición a la izquierda (arreglo[i] = arreglo[i+1]).
3. **Actualización:** Se reduce el contador elementos--.

Métodos de Consulta

- **mostrar():** Un bucle simple que imprime los datos desde el índice 0 hasta el número de elementos actuales.
- **estaVacia() y estaLlena():** Comparan el contador de elementos contra 0 o contra la capacidad total.



Que es lo que imprime y explicación:

Inicio y Primera Inserción:

- Se muestra una Capacidad: 3.
- Se selecciona la opción 1 (Agregar). Se ingresa el número 4.
- Imprime: "Elemento agregado.".

Segunda Inserción:

- Se selecciona la opción 1 (Agregar). Se ingresa el número 1.
- Imprime: "Elemento agregado.".

Intento de Desborde:

- Se selecciona la opción 1 (Agregar) nuevamente. Se ingresa el número 4.
- Al intentar agregar un cuarto elemento, el programa detecta que se alcanzó el límite y lanza el mensaje: "La lista esta llena.".

Consulta de Contenido (Opción 3):

- Imprime: Contenido: 4 1 4. (Nota: Aunque el último intento falló en la lógica de "llenado", los datos previos permanecen intactos).

Consulta de Tamaño (Opción 6):

- Imprime: Tamaño: 3.

Errores y Soluciones: no hubo errores.

Nuevo tipo de dato

main.cpp Estructura.h Lista.h

```cpp
#ifndef LISTA_H
#define LISTA_H

#include <iostream>
#include <string>
#include "Estructura.h"

using namespace std;

class Persona {
public:
    string nombre;
    int edad;

    Persona() {
        nombre = "";
        edad = 0;
    }

    Persona(string n, int e) {
        nombre = n;
        edad = e;
    }
};

class Lista : public Estructura {
private:
    Persona* arreglo;
    int capacidad;
    int elementos;

public:
    Lista(int cap) {
        capacidad = cap;
        elementos = 0;
        arreglo = new Persona[cap];
    }

    void agregar() {
        if (!estaLlena()) {
            string nombre;
            int edad;

            cout << "Nombre: ";
            cin >> nombre;
            cout << "Edad: ";
            cin >> edad;

            arreglo[elementos] = Persona(nombre, edad);
            elementos++;

            cout << "Persona agregada.\n";
        } else {
            cout << "La lista esta llena.\n";
        }
    }

    void quitar() {
        if (!estaVacia()) {
            int pos;
            cout << "Posicion a eliminar (0 a " << elementos-1 << "): ";
            cin >> pos;

            if (pos >= 0 && pos < elementos) {
                cout << "Eliminando: "
                     << arreglo[pos].nombre
                     << " (" << arreglo[pos].edad << ")\n";

                for (int i = pos; i < elementos - 1; i++) {
                    arreglo[i] = arreglo[i + 1];
                }
                elementos--;
            } else {
                cout << "Posicion invalida.\n";
            }
        } else {
            cout << "La lista esta vacia.\n";
        }
    }

    void mostrar() {
        if (estaVacia()) {
            cout << "La lista esta vacia.\n";
            return;
        }

        cout << "Contenido:\n";
        for (int i = 0; i < elementos; i++) {
            cout << arreglo[i].nombre
                 << " - " << arreglo[i].edad << endl;
        }
    }

    bool estaVacia() {
        return elementos == 0;
    }

    bool estaLlena() {
        return elementos == capacidad;
    }

    int tamanio() {
        return elementos;
    }
};

#endif
```

1. Objetivo del Código

El objetivo principal es demostrar cómo una estructura de datos (en este caso, una **Lista**) puede almacenar tipos de datos complejos (objetos de la clase Persona) en lugar de simples enteros. Permite gestionar una colección dinámica de registros donde cada elemento tiene múltiples atributos (nombre y edad), ofreciendo las operaciones básicas de insertar, eliminar por posición y listar el contenido.

Explicación del Código

La Clase Persona

Es la unidad básica de datos:

- **Atributos:** Tiene un string nombre y un int edad.
- **Constructores:** Incluye un constructor por defecto (inicializa en vacío/cero) y un constructor con parámetros para asignar valores específicos al crear una persona.

Atributos de la Clase Lista

- **Persona* arreglo:** Es un puntero a objetos de tipo Persona. Esto permite crear un arreglo dinámico de objetos en la memoria.
- **capacidad y elementos:** Controlan el tamaño máximo del arreglo y cuántas personas hay registradas actualmente.

Método agregar()

1. Verifica si hay espacio disponible.
2. Solicita al usuario el nombre y la edad por consola.
3. **Instanciación:** Crea un objeto Persona y lo guarda en la posición indicada por el contador elementos.
4. Incrementa el contador para la siguiente inserción.

Método quitar()

Es el método más importante para la gestión de memoria:

1. Pide al usuario la **posición** (índice) que desea borrar.
2. **Corrimiento:** Si la posición es válida, utiliza un bucle for para mover todas las personas a la izquierda una posición, "aplastando" el dato que se quería eliminar.
3. Decrementa elementos para reflejar que la lista es más corta.

Métodos de Consulta

- **mostrar():** Recorre el arreglo e imprime el nombre y la edad de cada persona almacenada.
- **estaVacia() / estaLlena():** Funciones lógicas que ayudan a prevenir errores al intentar borrar en listas vacías o agregar en listas saturadas.

```
Capacidad: 1

--- MENU ---
1. Agregar
2. Quitar
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
1
Nombre: Jimena
Edad: 13
Persona agregada.

--- MENU ---
1. Agregar
2. Quitar
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
1
La lista esta llena.
```

Que es lo que imprime y explicación:

1. **Estado Inicial:**

Muestra Capacidad: 1. Esto indica que el arreglo solo tiene espacio para un objeto de tipo Persona.

2. **Primera Inserción (Exitosa):**
   - Se selecciona la opción 1 (Agregar).
   - El programa pide datos: Nombre: Jimena y Edad: 13.
   - Imprime: Persona agregada..
3. **Segundo Intento (Fallo por Límite):**
   - Se selecciona de nuevo la opción 1 (Agregar).
   - Como la capacidad era de solo 1 y ya está ocupada, el programa ejecuta la validación estaLlena().
   - Imprime: La lista esta llena..

Errores y Soluciones: no hubo errores.

PILA (manera dinámica por medio de punteros al mismo tipo de dato)

Dato DB

main.cpp Estructura.h Pila.h

```cpp
#ifndef PILA_H
#define PILA_H

#include <iostream>
#include "Estructura.h"

using namespace std;

// ?? Nodo
struct Nodo {
    int dato;
    Nodo* siguiente;
};

class Pila : public Estructura {
private:
    Nodo* tope;
    int contador;

public:
    Pila() {
        tope = NULL;
        contador = 0;
    }

    void agregar() { // push
        int valor;
        cout << "Ingresa un numero: ";
        cin >> valor;

        Nodo* nuevo = new Nodo;
        nuevo->dato = valor;
        nuevo->siguiente = tope;

        tope = nuevo;
        contador++;
```

```cpp
        cout << "Elemento agregado.\n";
    }

    void quitar() { // pop
        if (!estaVacia()) {
            Nodo* temp = tope;

            cout << "Elemento eliminado: " << temp->dato << endl;

            tope = tope->siguiente;
            delete temp;
            contador--;
        } else {
            cout << "La pila esta vacia.\n";
        }
    }

    void mostrar() {
        if (estaVacia()) {
            cout << "La pila esta vacia.\n";
            return;
        }

        Nodo* aux = tope;
        cout << "Contenido (tope a base): ";
        while (aux != NULL) {
            cout << aux->dato << " ";
            aux = aux->siguiente;
        }
        cout << endl;
    }

    bool estaVacia() {
        return tope == NULL;
```

```cpp
    }

    bool estaLlena() {
        return false; // dinámica
    }

    int tamanio() {
        return contador;
    }
};

#endif
```

Objetivo del Código

El objetivo es crear una estructura de datos tipo **LIFO** (*Last In, First Out* - El último en entrar es el primero en salir). Se utiliza para gestionar datos de forma vertical, donde solo tienes acceso al elemento que está en la parte superior (el **tope**). Al ser dinámica, su objetivo es permitir que la pila crezca tanto como la memoria de la computadora lo permita, sin definir un tamaño máximo fijo desde el principio.

Explicación del Código

La Estructura Nodo

Es el bloque de construcción de la pila.

- **int dato**: El valor numérico que guardamos.
- **Nodo* siguiente**: Un puntero que "apunta" al nodo que está debajo de él. Esto crea la cadena que une toda la pila.

Atributos de la Clase Pila

- **Nodo* tope**: Es el puntero más importante; siempre guarda la dirección del elemento que está hasta arriba.
- **int contador**: Lleva el registro de cuántos elementos hay en la pila.

Método agregar() / Push

1. Se crea un nuevo nodo en memoria.
2. Se le asigna el valor ingresado por el usuario.
3. **La conexión**: El puntero siguiente del nuevo nodo apunta a lo que antes era el tope.
4. **Actualización**: El nuevo nodo se convierte ahora en el nuevo tope.

Método quitar() / Pop

1. Verifica si la pila no está vacía.
2. Usa un puntero temporal (temp) para apuntar al nodo del tope actual.
3. Mueve el tope al siguiente elemento (el de abajo).
4. **Liberación de memoria**: Usa delete temp para borrar el nodo de la memoria y evitar fugas de memoria (*memory leaks*).

Método mostrar()

Recorre la pila desde el tope hasta la base usando un puntero auxiliar (aux). Mientras aux no sea NULL, imprime el dato y salta al siguiente nodo.

Método estaLlena()

Nota que este método siempre devuelve false. Esto es porque, al ser una estructura dinámica, técnicamente nunca está "llena" (a menos que te quedes sin RAM en la PC).

```
--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 1
Ingresa un numero: 4
Elemento agregado.

--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 1
Ingresa un numero: 6
Elemento agregado.
```

```
--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 3
Contenido (tope a base): 6 4

--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 1
Ingresa un numero: 3
Elemento agregado.
```

```
--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 3
Contenido (tope a base): 3 6 4
```

Que es lo que imprime y explicación:

Primeras Inserciones:

- Se selecciona la opción 1 (Push) e ingresas el número 4.
- Se selecciona la opción 1 (Push) de nuevo e ingresas el número 6.
- El programa confirma en ambos casos: "Elemento agregado.".

Primera Consulta de Contenido:

- Al seleccionar la opción 3 (Mostrar), el programa imprime: Contenido (tope a base): 6 4.
- Nota que el 6 aparece primero porque fue el último en entrar (está en el "tope").

Tercera Inserción:

- Se selecciona la opción 1 (Push) e ingresas el número 3.

Consulta Final:

- Al seleccionar nuevamente la opción 3 (Mostrar), imprime: Contenido (tope a base): 3 6 4.

Errores y Soluciones: no hubo errores.

Nuevo tipo de dato

main.cpp | Estructura.h | Pila.h

```cpp
#ifndef PILA_H
#define PILA_H

#include <iostream>
#include <string>
#include "Estructura.h"

using namespace std;

//Tipo de dato
class Persona {
public:
    string nombre;
    int edad;

    Persona() {
        nombre = "";
        edad = 0;
    }

    Persona(string n, int e) {
        nombre = n;
        edad = e;
    }
};

//Nodo
struct Nodo {
    Persona dato;
    Nodo* siguiente;
};

class Pila : public Estructura {
private:
    Nodo* tope;
    int contador;

public:
    Pila() {
        tope = NULL;
        contador = 0;
    }

    void agregar() {
        string nombre;
        int edad;

        cout << "Nombre: ";
        cin >> nombre;
        cout << "Edad: ";
        cin >> edad;

        Nodo* nuevo = new Nodo;
        nuevo->dato = Persona(nombre, edad);
        nuevo->siguiente = tope;

        tope = nuevo;
        contador++;

        cout << "Persona agregada.\n";
    }

    void quitar() {
        if (!estaVacia()) {
            Nodo* temp = tope;

            cout << "Eliminando: "
                 << temp->dato.nombre
                 << " (" << temp->dato.edad << ")\n";

            tope = tope->siguiente;
            delete temp;
            contador--;
        } else {
            cout << "La pila esta vacia.\n";
        }
    }

    void mostrar() {
        if (estaVacia()) {
            cout << "La pila esta vacia.\n";
            return;
        }

        Nodo* aux = tope;

        cout << "Contenido:\n";
        while (aux != NULL) {
            cout << aux->dato.nombre
                 << " - " << aux->dato.edad << endl;
            aux = aux->siguiente;
        }
    }

    bool estaVacia() {
        return tope == NULL;
    }

    bool estaLlena() {
        return false;
    }

    int tamanio() {
        return contador;
    }
};

#endif
```

Objetivo del Código

El objetivo principal es implementar una **Pila Dinámica de Objetos**.

- **Pila (Stack):** Sigue la lógica **LIFO** (*Last In, First Out*), donde el último registro de persona que agregas es el primero que se elimina o se muestra.
- **Dinámica:** No tiene un tamaño fijo (como los arreglos). Utiliza nodos y punteros para crecer indefinidamente mientras haya memoria disponible.
- **De Objetos:** A diferencia de una pila de números simples, cada elemento de esta pila es un objeto de la clase Persona, permitiendo almacenar múltiples datos (nombre y edad) en un solo nodo.

Explicación del Código

Clase Persona

Es el molde para los datos que queremos guardar.

- Define dos atributos: nombre y edad.
- Incluye dos constructores: uno que inicializa los valores en vacío/cero y otro que recibe parámetros para asignar datos específicos al momento de crear el objeto.

Estructura Nodo

Es el contenedor físico dentro de la memoria.

- **Persona dato:** Aquí se guarda el objeto persona completo.
- **Nodo* siguiente:** Es un puntero que guarda la dirección del nodo que está "debajo", conectando toda la pila.

Método agregar()

Este es el método **Push**:

1. Pide al usuario el nombre y la edad.
2. Crea un nuevo nodo en memoria dinámica (new Nodo).
3. **La conexión clave:** El puntero siguiente del nuevo nodo apunta hacia donde esté el actual tope.
4. **Actualización:** El puntero tope se mueve para apuntar al nuevo nodo, que ahora es la cima de la pila.

Método quitar()

Este es el método **Pop**:

1. Verifica que la pila no esté vacía.

2. Usa un puntero temporal (temp) para no perder la dirección del nodo que se va a borrar.
3. Mueve el tope hacia el nodo de abajo (tope->siguiente).
4. Ejecuta delete temp para liberar la memoria del nodo eliminado.

Método mostrar()

Recorre la pila desde arriba hacia abajo.

- Usa un puntero auxiliar (aux) que comienza en el tope.
- Mientras aux no llegue al final de la cadena (NULL), imprime el nombre y la edad de la persona guardada en ese nodo y salta al siguiente.

```
--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 1
Nombre: Carlos
Edad: 23
Persona agregada.

--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 1
Nombre: Miguel
Edad: 25
Persona agregada.
```

```
--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 3
Contenido:
Miguel - 25
Carlos - 23
```

Que es lo que imprime y explicación:

Primera Inserción:

- Se selecciona la Opcion: 1 (Push).
- Se ingresan los datos: Nombre: Carlos, Edad: 23.

- El programa imprime: Persona agregada..

Segunda Inserción:

- Se selecciona nuevamente la Opcion: 1 (Push).
- Se ingresan los datos: Nombre: Miguel, Edad: 25.
- El programa imprime: Persona agregada..

Consulta de Contenido:

- Se selecciona la Opcion: 3 (Mostrar).
- El programa imprime el encabezado Contenido: seguido de la lista en orden de tope a base:
    - Miguel - 25
    - Carlos - 23

Errores y Soluciones: no hubo errores.

COLA (manera dinámica por medio de punteros al mismo tipo de dato)

Dato DB

main.cpp Estructura.h Cola.h

```cpp
#ifndef COLA_H
#define COLA_H

#include <iostream>
#include "Estructura.h"

using namespace std;

//  Nodo con int
struct Nodo {
    int dato;
    Nodo* siguiente;
};

class Cola : public Estructura {
private:
    Nodo* frente;
    Nodo* fin;
    int contador;

public:
    Cola() {
        frente = NULL;
        fin = NULL;
        contador = 0;
    }

    void agregar() override {
        int valor;
        cout << "Ingresa un numero: ";
        cin >> valor;

        Nodo* nuevo = new Nodo;
        nuevo->dato = valor;
        nuevo->siguiente = NULL;

        if (estaVacia()) {
            frente = fin = nuevo;
        } else {
            fin->siguiente = nuevo;
            fin = nuevo;
        }

        contador++;
        cout << "Elemento agregado.\n";
    }

    void quitar() override {
        if (!estaVacia()) {
            Nodo* temp = frente;
            cout << "Elemento eliminado: " << temp->dato << endl;

            frente = frente->siguiente;
            delete temp;

            if (frente == NULL)
                fin = NULL;
        } else {
            cout << "La cola esta vacia.\n";
        }
    }

    void mostrar() override {
        if (estaVacia()) {
            cout << "La cola esta vacia.\n";
            return;
        }

        Nodo* aux = frente;
        cout << "Contenido: ";
        while (aux != NULL) {
            cout << aux->dato << " ";
            aux = aux->siguiente;
        }
        cout << endl;
    }

    bool estaVacia() override {
        return frente == NULL;
    }

    bool estaLlena() override {
        return false; // dinámica
    }

    int tamanio() override {
        return contador;
    }
};

#endif
```

Objetivo del Código

El objetivo es gestionar una lista de números enteros de forma dinámica, permitiendo agregar elementos al final de la fila y eliminarlos desde el principio. Al ser dinámica, la cola crece y se reduce en memoria según sea necesario, utilizando punteros para conectar los nodos.

Explicación del Código

La Estructura Nodo

Es el bloque básico de construcción.

- **int dato**: Almacena el valor numérico.
- **Nodo* siguiente**: Un puntero que guarda la dirección del próximo elemento en la fila.

Atributos de la Clase Cola

- **frente**: Puntero al primer elemento (donde se eliminan datos).
- **fin**: Puntero al último elemento (donde se insertan datos).
- **contador**: Lleva el registro de cuántos elementos hay en la cola.

Método agregar() / Enqueue

Es el proceso de formarse en la fila:

1. Crea un nuevo nodo en memoria (new Nodo).
2. Si la cola está vacía, tanto el frente como el fin apuntan al nuevo nodo.
3. Si ya hay elementos, el nodo que estaba al fin ahora apunta al nuevo, y luego actualizamos el puntero fin para que sea el nuevo nodo.

Método quitar() / Dequeue

Es el proceso de salir de la fila después de ser atendido:

1. Verifica si hay elementos (!estaVacia()).
2. Usa un puntero temporal temp para guardar la dirección del frente.
3. Mueve el puntero frente al siguiente nodo de la fila.
4. Libera la memoria con delete temp. Si la cola queda vacía tras esto, ajusta el puntero fin a NULL.

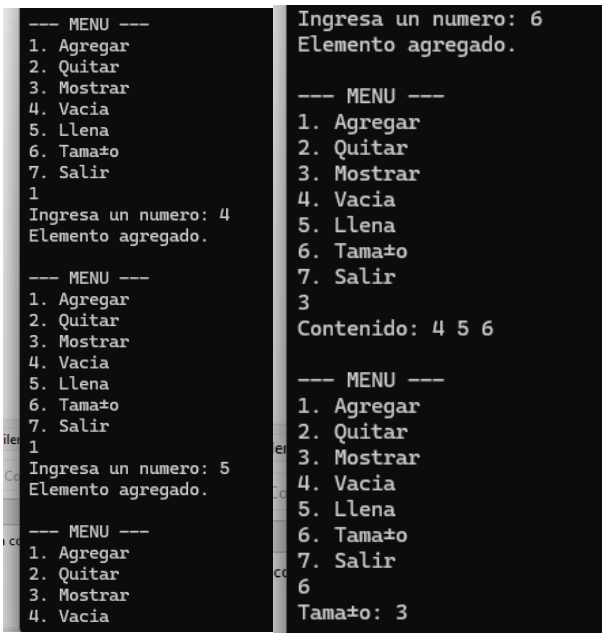
Método mostrar()

Recorre la cola para imprimir los valores:

- Utiliza un puntero auxiliar (aux) para no perder la referencia del frente original mientras "camina" por toda la estructura hasta llegar a NULL.

Diferencia con la Pila

A diferencia de la Pila (donde todo sucede en el "tope"), aquí usamos dos "anclas": una para saber quién sigue (**frente**) y otra para saber dónde poner al que llega (**fin**). Esto garantiza que el orden de llegada se respete estrictamente.



Que es lo que imprime y explicación:

Esta ejecución muestra el comportamiento de una Cola Dinámica basada en la lógica FIFO (*First In, First Out*), donde los elementos se mantienen en el orden exacto en el que fueron ingresados.

¿Qué imprime el programa?

1. **Primeras Inserciones:**
   - Se selecciona la opción 1 (Agregar) e ingresas el número 4. El sistema imprime: "Elemento agregado.".
   - Se selecciona la opción 1 (Agregar) e ingresas el número 5. El sistema imprime: "Elemento agregado.".
2. **Tercera Inserción:**
   - Se selecciona la opción 1 (Agregar) e ingresas el número 6. El sistema imprime: "Elemento agregado.".
3. **Consulta de Contenido:**
   - Al seleccionar la opción 3 (Mostrar), el programa imprime: Contenido: 4 5 6.

- Nota que el 4 aparece primero porque fue el primero en entrar (está en el "frente").

4. **Consulta de Tamaño:**
   - Al seleccionar la opción 6 (Tamaño), el programa imprime: Tamaño: 3.

Errores y Soluciones: no hubo errores.

Nuevo Tipo de dato

main.cpp | Estructura.h | Cola.h

```cpp
#ifndef COLA_H
#define COLA_H

#include <iostream>
#include <string>
#include "Estructura.h"

using namespace std;

// ?? Tipo de dato propio
class Persona {
public:
    string nombre;
    int edad;

    Persona() {
        nombre = "";
        edad = 0;
    }

    Persona(string n, int e) {
        nombre = n;
        edad = e;
    }
};

// ?? Nodo
struct Nodo {
    Persona dato;
    Nodo* siguiente;
};

class Cola : public Estructura {
private:
    Nodo* frente;
    Nodo* fin;
    int contador;

public:
    Cola() {
        frente = NULL;
        fin = NULL;
        contador = 0;
    }

    void agregar() {
        string nombre;
        int edad;

        cout << "Nombre: ";
        cin >> nombre;
        cout << "Edad: ";
        cin >> edad;

        Nodo* nuevo = new Nodo;
        nuevo->dato = Persona(nombre, edad);
        nuevo->siguiente = NULL;

        if (estaVacia()) {
            frente = nuevo;
            fin = nuevo;
        } else {
            fin->siguiente = nuevo;
            fin = nuevo;
        }

        contador++;
        cout << "Persona agregada.\n";
    }

    void quitar() {
        if (!estaVacia()) {
            Nodo* temp = frente;

            cout << "Eliminando: "
                 << temp->dato.nombre
                 << " (" << temp->dato.edad << ")\n";

            frente = frente->siguiente;
            delete temp;
            contador--;

            if (frente == NULL) {
                fin = NULL;
            }
        } else {
            cout << "La cola esta vacia.\n";
        }
    }

    void mostrar() {
        if (estaVacia()) {
            cout << "La cola esta vacia.\n";
            return;
        }

        Nodo* aux = frente;
        cout << "Contenido:\n";

        while (aux != NULL) {
            cout << aux->dato.nombre
                 << " - " << aux->dato.edad << endl;
            aux = aux->siguiente;
        }
    }

    bool estaVacia() {
        return frente == NULL;
    }

    bool estaLlena() {
        return false; // dinámica
    }

    int tamanio() {
        return contador;
    }
};

#endif
```

Objetivo del Código

El objetivo es crear una estructura de datos tipo FIFO (*First In, First Out*) que almacene objetos de la clase Persona. Esto simula una fila de la vida real donde la primera persona en llegar es la primera en ser atendida (o eliminada de la lista), gestionando la memoria de forma dinámica mediante nodos y punteros.

Explicación del Código

Clase Persona

Es la unidad de datos. Cada "persona" tiene un nombre y una edad.

- Constructores: Tiene uno por defecto (vacío) y uno con parámetros para asignar valores rápidamente al crear el objeto.

Estructura Nodo

Es el "eslabón" de la cadena.

- Persona dato: Guarda el objeto persona.
- Nodo* siguiente: Es el puntero que conecta con la persona que está detrás en la fila.

Clase Cola (Atributos y Constructor)

- frente y fin: Son punteros clave. El frente sabe quién sigue y el fin sabe dónde poner al nuevo integrante.
- Constructor: Inicializa todo en NULL y el contador en 0 porque la cola empieza vacía.

Método agregar()

Es la operación Enqueue:

1. Pide los datos por teclado.
2. Crea un nuevo nodo (new Nodo).
3. Lógica de inserción:
    - Si la cola está vacía, la nueva persona es tanto el frente como el fin.
    - Si ya hay gente, el que estaba al final (fin->siguiente) ahora apunta al nuevo, y actualizamos el puntero fin a esta nueva persona.

Método quitar()

Es la operación Dequeue:

1. Verifica que no esté vacía.
2. Respalda el frente en un puntero temp para no perderlo.
3. Mueve el frente al siguiente nodo.

4. delete temp: Borra físicamente el nodo de la memoria para evitar fugas (*memory leaks*).

Método mostrar()

Recorre la fila de principio a fin.

- Utiliza un puntero aux para "caminar" por los nodos imprimiendo el nombre y la edad de cada persona hasta llegar a NULL.

```
--- MENU ---
1. Agregar persona
2. Quitar persona
3. Mostrar contenido
4. Verificar si esta vacia
5. Verificar si esta llena
6. Mostrar tama±o
7. Salir
Opcion: 1
Nombre: Fernando
Edad: 12
Persona agregada.

--- MENU ---
1. Agregar persona
2. Quitar persona
3. Mostrar contenido
4. Verificar si esta vacia
5. Verificar si esta llena
6. Mostrar tama±o
7. Salir
Opcion: 1
Nombre: Fernanda
Edad: 13
Persona agregada.
```

```
--- MENU ---
1. Agregar persona
2. Quitar persona
3. Mostrar contenido
4. Verificar si esta vacia
5. Verificar si esta llena
6. Mostrar tama±o
7. Salir
Opcion: 3
Contenido:
Fernando - 12
Fernanda - 13
```

Que es lo que imprime y explicación:

Registro de datos:

- Se selecciona la Opción 1 para agregar a Fernando con 12 años. El sistema confirma con "Persona agregada.".
- Se selecciona nuevamente la Opción 1 para agregar a Fernanda con 13 años. El sistema confirma con "Persona agregada.".

Muestra de contenido:

- Al seleccionar la Opción 3 (Mostrar contenido), el programa imprime el encabezado Contenido: y los datos en el orden en que fueron ingresados:
    - Fernando - 12 (al frente de la cola).
    - Fernanda - 13 (al final de la cola).

Errores y Soluciones: no hubo errores.

LISTA (manera dinámica por medio de punteros al mismo tipo de dato)

Dato DB

```cpp
    void quitar() {
        if (!estaVacia()) {
            int pos;
            cout << "Posicion a eliminar (0 a " << elementos-1 << "): ";
            cin >> pos;

            if (pos >= 0 && pos < elementos) {
                cout << "Eliminado: " << arreglo[pos] << endl;

                for (int i = pos; i < elementos - 1; i++) {
                    arreglo[i] = arreglo[i + 1];
                }

                elementos--;
            } else {
                cout << "Posicion invalida.\n";
            }
        } else {
            cout << "La lista esta vacia.\n";
        }
    }

    void mostrar() {
        if (estaVacia()) {
            cout << "La lista esta vacia.\n";
            return;
        }

        cout << "Contenido: ";
        for (int i = 0; i < elementos; i++) {
            cout << arreglo[i] << " ";
        }
        cout << endl;
    }

    bool estaVacia() {
```

```cpp
        return elementos == 0;
    }

    bool estaLlena() {
        return elementos == capacidad;
    }

    int tamanio() {
        return elementos;
    }
};

#endif
```

Objetivo del Código

El objetivo es gestionar una colección de elementos donde se permite la **eliminación en cualquier posición** y el recorrido de los datos. Al ser una estructura estática, el orden de los elementos se mantiene mediante el desplazamiento físico de los datos dentro del arreglo cuando uno es removido.

Explicación del Código

Método quitar()

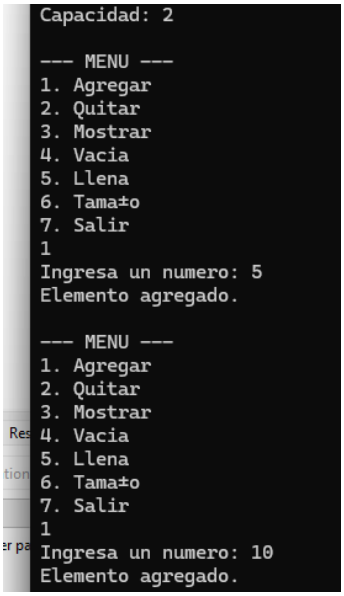
Esta es la función más compleja porque debe mantener la integridad de la lista:

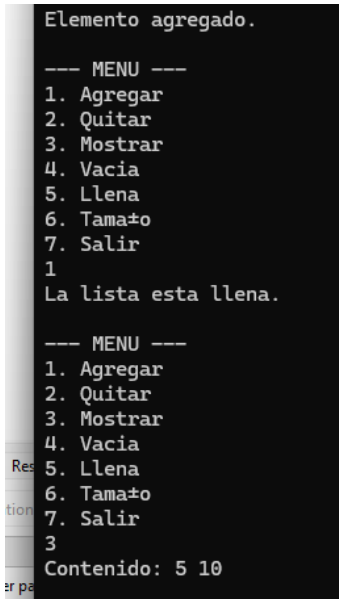
1. **Validación de Vacío**: Primero verifica que haya elementos para borrar usando estaVacia().
2. **Entrada de Posición**: Pide al usuario el índice exacto
3. **Validación de Rango**: Se asegura de que la posición exista dentro del número actual de elementos.
4. **El "Corrimiento":** Esta es la clave de las listas estáticas.
    - Si borras el elemento en la posición 2, el elemento de la posición debe moverse a la 2, el de la 4 a la 3, y así sucesivamente.
    - El ciclo for recorre desde la posición eliminada hasta el final, moviendo cada dato una casilla hacia la izquierda (arreglo[i] = arreglo[i + 1]).
5. **Actualización**: Se resta 1 a la variable elementos para indicar que la lista es más pequeña.

Método mostrar()

Es un recorrido lineal estándar:

- Si la lista no está vacía, usa un ciclo for que va desde el índice 0 hasta elementos - 1.
- Imprime cada valor guardado en el arreglo seguido de un espacio.





Que es lo que imprime y explicación:

Configuración Inicial:

- En la parte superior se observa que la Capacidad es 2.

Inserciones Exitosas:

- Seleccionas la Opción 1 e ingresas el número 5. Imprime: "Elemento agregado.".
- Seleccionas la Opción 1 e ingresas el número 10. Imprime: "Elemento agregado.".

Límite de Capacidad

- Intentas agregar un tercer elemento (Opción 1). Como la capacidad es 2 y ya está llena, el programa imprime: "La lista esta llena.".

Consulta de Contenido:

- Seleccionas la Opción 3 (Mostrar). El programa imprime: Contenido: 5 10.

Errores y Soluciones: no hubo errores.

Tipo de dato

main.cpp Estructura.h Lista.h

```cpp
#ifndef LISTA_H
#define LISTA_H

#include <iostream>
#include <string>
#include "Estructura.h"

using namespace std;

class Persona {
public:
    string nombre;
    int edad;

    Persona() {
        nombre = "";
        edad = 0;
    }

    Persona(string n, int e) {
        nombre = n;
        edad = e;
    }
};

class Lista : public Estructura {
private:
    Persona* arreglo;
    int capacidad;
    int elementos;

public:
    Lista(int cap) {
        capacidad = cap;
        elementos = 0;
        arreglo = new Persona[cap];
    }

    void agregar() {
        if (!estaLlena()) {
            string nombre;
            int edad;

            cout << "Nombre: ";
            cin >> nombre;
            cout << "Edad: ";
            cin >> edad;

            arreglo[elementos] = Persona(nombre, edad);
            elementos++;

            cout << "Persona agregada.\n";
        } else {
            cout << "La lista esta llena.\n";
        }
    }

    void quitar() {
        if (!estaVacia()) {
            int pos;
            cout << "Posicion a eliminar (0 a " << elementos-1 << "): ";
            cin >> pos;

            if (pos >= 0 && pos < elementos) {
                cout << "Eliminando: "
                     << arreglo[pos].nombre
                     << " (" << arreglo[pos].edad << ")\n";

                for (int i = pos; i < elementos - 1; i++) {
                    arreglo[i] = arreglo[i + 1];
                }
                elementos--;
            } else {
                cout << "Posicion invalida.\n";
            }
        } else {
            cout << "La lista esta vacia.\n";
        }
    }

    void mostrar() {
        if (estaVacia()) {
            cout << "La lista esta vacia.\n";
            return;
        }

        cout << "Contenido:\n";
        for (int i = 0; i < elementos; i++) {
            cout << arreglo[i].nombre
                 << " - " << arreglo[i].edad << endl;
        }
    }

    bool estaVacia() {
        return elementos == 0;
    }

    bool estaLlena() {
        return elementos == capacidad;
    }

    int tamanio() {
        return elementos;
    }
};

#endif
```

Objetivo del Código

El objetivo principal es crear un contenedor de datos con una **capacidad fija** que permita almacenar objetos complejos (nombre y edad). A diferencia de una pila o una cola, esta "Lista" permite **eliminar elementos en cualquier posición** intermedia, manteniendo el orden de los datos mediante desplazamientos de memoria.

Explicación del Código

Clase Persona

Es la base de los datos que vamos a guardar.

- Tiene dos atributos: nombre (string) y edad (int).
- Incluye un **constructor con parámetros** (línea 20) que facilita la creación de un objeto Persona con datos específicos en una sola línea.

Clase Lista (Atributos y Constructor)

- **Persona* arreglo**: Es un puntero que se usa para crear un arreglo dinámico de objetos Persona.
- **Constructor**: Recibe una capacidad (cap). Reserva el espacio exacto en memoria usando new Persona[cap].

Método agregar()

Inserta una nueva persona al final de la lista.

- Verifica si hay espacio con !estaLlena().
- Crea un objeto Persona con los datos ingresados y lo guarda en la posición indicada por el contador elementos.
- Incrementa el contador para que la siguiente persona se guarde en la casilla de al lado.

Método quitar()

Permite borrar a alguien de **cualquier parte** de la lista (no solo del final).

- Pide una **posición específica** al usuario (ejemplo: borrar a la persona en el índice 1).
- **El Corrimiento;** Esta es la parte vital. Si eliminas a alguien del medio, los que están a la derecha deben "recorrerse" una casilla a la izquierda para que no quede un hueco vacío en el arreglo.
- Se decrementa elementos para actualizar el tamaño actual.

Método mostrar()

Recorre el arreglo desde el índice 0 hasta elementos - 1.

- Imprime el nombre y la edad de cada persona guardada.

```
Capacidad: 1

--- MENU ---
1. Agregar
2. Quitar
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
1
Nombre: Jaret
Edad: 20
Persona agregada.

--- MENU ---
1. Agregar
2. Quitar
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
1
La lista esta llena.
```

Que es lo que imprime y explicación:

Estado Inicial:

- El programa indica una Capacidad: 1 al inicio.

Primera Inserción:

- Seleccionas la Opción 1.
- Ingresas el nombre Jaret y la edad 20.
- El programa imprime: "Persona agregada.".

Intento de Desbordamiento:

- Seleccionas nuevamente la Opción 1.
- Como la capacidad máxima es 1 y ya hay un elemento guardado, el programa detecta que no hay espacio y directamente imprime: "La lista esta llena.".

Errores y Soluciones: no hubo errores.

PILA (manera dinámica por medio de las librerías o clases proporcionadas por el IDE)

Dato DB

main.cpp | Estructura.h | Pila.h

```cpp
#ifndef PILA_H
#define PILA_H

#include <iostream>
#include <stack>
#include "Estructura.h"

using namespace std;

class Pila : public Estructura {
private:
    stack<int> pila;
    int capacidad;

public:
    Pila(int cap) {
        capacidad = cap;
    }

    void agregar() {
        if (!estaLlena()) {
            int valor;
            cout << "Ingresa un numero: ";
            cin >> valor;

            pila.push(valor);
            cout << "Elemento agregado.\n";
        } else {
            cout << "La pila esta llena.\n";
        }
    }

    void quitar() {
        if (!estaVacia()) {
            cout << "Elemento eliminado: " << pila.top() << endl;
            pila.pop();
            pila.pop();
        } else {
            cout << "La pila esta vacia.\n";
        }
    }

    void mostrar() {
        if (estaVacia()) {
            cout << "La pila esta vacia.\n";
            return;
        }

        stack<int> temp = pila;

        cout << "Contenido (tope a base): ";
        while (!temp.empty()) {
            cout << temp.top() << " ";
            temp.pop();
        }
        cout << endl;
    }

    bool estaVacia() {
        return pila.empty();
    }

    bool estaLlena() {
        return pila.size() == capacidad;
    }

    int tamanio() {
        return pila.size();
    }
};

#endif
```

Objetivo del Código

El objetivo es crear una estructura de datos tipo Pila con una **capacidad limitada** definida por el usuario. Permite gestionar una colección de números enteros, asegurando que las inserciones y extracciones ocurran siempre por el "tope" de la pila, respetando el límite máximo de elementos para evitar un desbordamiento de memoria.

Explicación del Código

Definición y Atributos

- **class Pila : public Estructura**: Define la clase Pila que hereda de una clase base llamada Estructura.
- **stack<int> pila**: Utiliza el contenedor estándar de C++ para manejar los datos.
- **int capacidad**: Variable para controlar el tamaño máximo permitido.
- **Constructor**: Inicializa la capacidad con el valor proporcionado al crear el objeto.

Método agregar()

Es la operación **Push**:

1. Verifica si la pila tiene espacio disponible con !estaLlena().

2. Si hay espacio, solicita un número al usuario y lo coloca en el tope usando pila.push(valor).
3. Si no hay espacio, imprime un mensaje de error indicando que la pila está llena.

Método quitar()

Es la operación **Pop**:

1. Verifica que la pila no esté vacía con !estaVacia().
2. Muestra el valor que está en el tope (pila.top()) y luego lo elimina con pila.pop().
3. Si la pila está vacía, avisa al usuario que no hay nada que eliminar.

Método mostrar()

Imprime los elementos de la pila desde el tope hasta la base:

- Para no destruir la pila original al mostrarla, crea una copia temporal: stack<int> temp = pila;.
- Usa un ciclo while para extraer y mostrar el tope de la copia temporal hasta que esta quede vacía.

Funciones Auxiliares

- **estaVacia()**: Retorna true si la pila interna no tiene elementos.
- **estaLlena()**: Compara el tamaño actual de la pila (pila.size()) con la capacidad establecida.
- **tamanio()**: Devuelve la cantidad de elementos presentes actualmente.

```
Capacidad de la pila: 0

--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 1
La pila esta llena.

--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: |
```

Que es lo que imprime y explicación:

Configuración Inicial: En la primera línea se lee Capacidad de la pila: 0. Esto significa que el objeto Pila fue instanciado con un valor de capacidad = 0.

La Acción: Seleccionas la Opcion: 1, que corresponde a la operación Push (agregar un elemento).

El Resultado: El programa imprime inmediatamente "La pila esta llena.".

Errores y Soluciones: no hubo errores.

Nuevo tipo de dato

main.cpp Estructura.h Pila.h

```cpp
#ifndef PILA_H
#define PILA_H

#include <iostream>
#include <stack>
#include <string>
#include "Estructura.h"

using namespace std;

class Persona {
public:
    string nombre;
    int edad;

    Persona() {
        nombre = "";
        edad = 0;
    }

    Persona(string n, int e) {
        nombre = n;
        edad = e;
    }
};

class Pila : public Estructura {
private:
    stack<Persona> pila;
    int capacidad;

public:
    Pila(int cap) {
        capacidad = cap;
    }

    void agregar() {
        if (!estaLlena()) {
            string nombre;
            int edad;

            cout << "Nombre: ";
            cin >> nombre;
            cout << "Edad: ";
            cin >> edad;

            pila.push(Persona(nombre, edad));
            cout << "Persona agregada.\n";
        } else {
            cout << "La pila esta llena.\n";
        }
    }

    void quitar() {
        if (!estaVacia()) {
            Persona p = pila.top();
            cout << "Eliminando: " << p.nombre << " (" << p.edad << ")\n";
            pila.pop();
        } else {
            cout << "La pila esta vacia.\n";
        }
    }

    void mostrar() {
        if (estaVacia()) {
            cout << "La pila esta vacia.\n";
            return;
        }

        stack<Persona> temp = pila;

        cout << "Contenido:\n";
        while (!temp.empty()) {
            cout << temp.top().nombre
                 << " - " << temp.top().edad << endl;
            temp.pop();
        }
    }

    bool estaVacia() {
        return pila.empty();
    }

    bool estaLlena() {
        return pila.size() == capacidad;
    }

    int tamanio() {
        return pila.size();
    }
};

#endif
```

Objetivo del Código

El objetivo es gestionar una colección de registros de personas utilizando la estructura de datos **LIFO** (*Last In, First Out*). El programa permite almacenar el nombre y la edad de varios individuos, asegurando que el último registro ingresado sea siempre el primero en ser procesado o eliminado, todo bajo un límite de **capacidad máxima** para controlar el uso de memoria.

Explicación del Código

Clase Persona

Es la unidad básica de información.

- **Atributos**: Define un string nombre y un int edad.
- **Constructores**: Tiene uno por defecto (vacío) y uno parametrizado que permite crear una persona y asignar sus datos rápidamente (línea 21).

Clase Pila (Atributos y Constructor)

- **stack<Persona> pila**: Utiliza el contenedor de la biblioteca estándar, pero ahora especializado para guardar objetos Persona.
- **int capacidad**: Almacena el límite máximo de elementos que puede contener la pila.

Método agregar()

Equivale a la operación **Push**:

1. Verifica si hay espacio disponible con estaLlena().
2. Si hay cupo, pide al usuario el nombre y la edad.
3. Crea un objeto Persona "al vuelo" y lo inserta en el tope con pila.push().

Método quitar()

Equivale a la operación **Pop**:

1. Verifica que la pila no esté vacía.
2. Obtiene la persona que está en el tope (pila.top()) para imprimir sus datos antes de borrarla.
3. Elimina físicamente el objeto de la pila con pila.pop().

Método mostrar()

Recorre la pila para ver quiénes están dentro:

- Crea una copia temporal (temp) para no vaciar la pila original mientras se imprime.

- Usa un ciclo while para mostrar el nombre y la edad de cada persona desde el tope hasta la base.

Funciones de Estado

- **estaVacia()**: Indica si la pila tiene cero elementos.
- **estaLlena()**: Compara el tamaño actual de la pila con la capacidad límite establecida en el constructor.
- **tamanio()**: Devuelve cuántas personas hay registradas actualmente.

Diferencia Clave

A diferencia de una pila de enteros, aquí manejas **datos estructurados**. Esto es muy común en sistemas reales (como una lista de espera o una pila de procesos en un sistema operativo) donde cada elemento tiene múltiples propiedades que deben viajar juntas.

```
--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 1
Nombre: Mariana
Edad: 30
Persona agregada.

--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 1
La pila esta llena.
```

```
--- MENU ---
1. Push
2. Pop
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
Opcion: 3
Contenido:
Mariana - 30
```

Que es lo que imprime y explicación:

Primera Inserción (Éxito):

- Seleccionas la Opcion: 1 (Push).
- Ingresas el nombre Mariana y la edad 30.
- El programa utiliza el constructor Persona(nombre, edad) para crear el objeto y lo guarda en el tope.
- Imprime: "Persona agregada.".

Segunda Inserción (Fallo):

- Intentas de nuevo la Opcion: 1.

- Como la capacidad es 1 y ya hay un elemento, el método estaLlena() devuelve true.
- Imprime: "La pila esta llena.".

Consulta de Datos:

- Seleccionas la Opcion: 3 (Mostrar).
- El programa imprime el encabezado "Contenido:" y los datos del objeto en el tope: Mariana - 30.

Errores y Soluciones: no hubo errores.

COLA (manera dinámica por medio de las librerías o clases proporcionadas por el IDE)

Dato DB

```cpp
#include <iostream>
#include "Cola.h"

using namespace std;

int main() {
    int cap;
    cout << "Capacidad de la cola: ";
    cin >> cap;

    Cola c(cap);

    int opcion, valor;

    do {
        cout << "\n--- MENU ---\n";
        cout << "1. Agregar elemento\n";
        cout << "2. Quitar elemento\n";
        cout << "3. Mostrar contenido\n";
        cout << "4. Verificar si esta vacia\n";
        cout << "5. Verificar si esta llena\n";
        cout << "6. Mostrar tamaño\n";
        cout << "7. Salir\n";
        cout << "Opcion: ";
        cin >> opcion;

        switch (opcion) {
        case 1:
            cout << "Ingresa un numero: ";
            cin >> valor;
            c.agregar(valor);
            break;

        case 2:
            c.quitar();
            break;
```

```cpp
        case 3:
            c.mostrar();
            break;

        case 4:
            cout << (c.estaVacia() ? "Esta vacia\n" : "No esta vacia\n");
            break;

        case 5:
            cout << (c.estaLlena() ? "Esta llena\n" : "No esta llena\n");
            break;

        case 6:
            cout << "Tamaño actual: " << c.tamanio() << endl;
            break;

        case 7:
            cout << "Saliendo...\n";
            break;

        default:
            cout << "Opcion invalida\n";
        }

    } while (opcion != 7);

    return 0;
}
```

Objetivo del Código

El objetivo principal es proporcionar una **interfaz de usuario de consola** para gestionar una cola de números enteros. Permite al usuario definir el tamaño máximo de la estructura y manipularla mediante un menú interactivo, aplicando el principio **FIFO** (*First In, First Out*: el primero en llegar es el primero en salir).

Explicación del Código

Configuración Inicial

#include "Cola.h": Importa la definición de la clase Cola (donde reside la lógica de cómo se guardan los datos).

- **int cap; cin >> cap;**: Lo primero que hace el programa es preguntar al usuario cuál será la **capacidad máxima** de la cola.
- **Cola c(cap);**: Crea el objeto c enviando esa capacidad al constructor para reservar el espacio necesario.

El Ciclo de Interacción

Utiliza un bucle do-while que mantiene el programa ejecutándose hasta que el usuario elija la opción **7 (Salir)**.

Funciones del Menú (Switch Cases)

Dentro del switch, cada "case" corresponde a una operación fundamental de las estructuras de datos:

- **Caso 1 (Agregar):** Solicita un número y llama a c.agregar(valor). Es la operación de **Encolar**.
- **Caso 2 (Quitar):** Llama a c.quitar(). Elimina el elemento que lleva más tiempo en la cola (**Desencolar**).
- **Caso 3 (Mostrar):** Imprime todos los elementos presentes actualmente en la cola sin eliminarlos.
- **Casos 4 y 5 (Verificaciones):**
    - Utiliza un **operador ternario** (? :) para imprimir un mensaje amigable indicando si la cola está totalmente vacía o al límite de su capacidad.
- **Caso 6 (Tamaño):** Muestra cuántos elementos hay guardados en ese momento exacto.

```
Capacidad de la cola: 1

--- MENU ---
1. Agregar elemento
2. Quitar elemento
3. Mostrar contenido
4. Verificar si esta vacia
5. Verificar si esta llena
6. Mostrar tama±o
7. Salir
Opcion:
1
Ingresa un numero: 3
Elemento agregado: 3

--- MENU ---
1. Agregar elemento
2. Quitar elemento
3. Mostrar contenido
4. Verificar si esta vacia
5. Verificar si esta llena
6. Mostrar tama±o
7. Salir
Opcion: 3
Contenido: 3
```

Que es lo que imprime y explicación:

Configuración inicial: El usuario define la **Capacidad de la cola: 1**. Esto significa que solo puede haber un elemento almacenado a la vez.

Operación de Agregado (Push/Enqueue):

- Se elige la **Opcion: 1**.
- Se ingresa el número **3**.
- El programa confirma: **Elemento agregado: 3**.

Operación de Visualización:

- Se elige la **Opcion: 3** (Mostrar contenido).
- El programa imprime: **Contenido: 3**.

Errores y Soluciones: no hubo errores.

Nuevo tipo de dato

main.cpp Estructura.h Cola.h

```cpp
#ifndef COLA_H
#define COLA_H

#include <iostream>
#include <queue>
#include <string>
#include "Estructura.h"

using namespace std;

// ?? Nuevo tipo de dato
class Persona {
public:
    string nombre;
    int edad;

    Persona() {}

    Persona(string n, int e) {
        nombre = n;
        edad = e;
    }
};

// ?? Clase Cola
class Cola : public Estructura {
private:
    queue<Persona> cola;
    int capacidad;

public:
    Cola(int cap) {
        capacidad = cap;
    }

    void agregar() override {
        if (!estaLlena()) {
            string nombre;
            int edad;

            cout << "Nombre: ";
            cin >> nombre;
            cout << "Edad: ";
            cin >> edad;

            Persona p(nombre, edad);
            cola.push(p);

            cout << "Persona agregada.\n";
        } else {
            cout << "La cola esta llena.\n";
        }
    }

    void quitar() override {
        if (!estaVacia()) {
            Persona p = cola.front();
            cout << "Eliminando: " << p.nombre << " (" << p.edad << ")\n";
            cola.pop();
        } else {
            cout << "La cola esta vacia.\n";
        }
    }

    void mostrar() override {
        if (estaVacia()) {
            cout << "La cola esta vacia.\n";
            return;
        }

        queue<Persona> temp = cola;
        cout << "Contenido:\n";

        while (!temp.empty()) {
            Persona p = temp.front();
            cout << p.nombre << " - " << p.edad << endl;
            temp.pop();
        }
    }

    bool estaVacia() override {
        return cola.empty();
    }

    bool estaLlena() override {
        return cola.size() == capacidad;
    }

    int tamanio() override {
        return cola.size();
    }
};

#endif
```

Objetivo del Código

El objetivo es implementar una estructura de datos de tipo **FIFO** (*First In, First Out*: el primero en entrar es el primero en salir) para administrar un registro de personas. El programa permite controlar el flujo de "atención" o almacenamiento, asegurando que se respete el orden de llegada y una capacidad máxima de memoria.

Explicación del Código

La Clase Persona

Es un contenedor de datos simple que representa a un individuo:

- **Atributos:** Tiene un string nombre y un int edad.
- **Constructores:** Incluye un constructor vacío y uno que asigna valores directamente, facilitando la creación de objetos "al vuelo".

La Clase Cola

Esta clase hereda de una clase base llamada Estructura y utiliza el contenedor estándar queue<Persona> para funcionar.

- **capacidad:** Es una variable privada que limita cuántos objetos puede tener la cola, evitando el desbordamiento de datos.
- **agregar() (Push):**
  - Verifica si la cola tiene espacio (!estaLlena()).
  - Pide el nombre y edad por teclado.
  - Crea una Persona p y la inserta al final de la fila con cola.push(p).
- **quitar() (Pop):**
  - Sigue la lógica FIFO: extrae al elemento que está al frente (cola.front()).
  - Muestra el nombre de quién está siendo eliminado antes de borrarlo definitivamente con cola.pop().
- **mostrar():**
  - Para no vaciar la cola original al imprimir, crea una copia temporal (queue<Persona> temp = cola).
  - Usa un ciclo while para recorrer la copia y mostrar a cada persona hasta que la cola temporal quede vacía.

Funciones de Control

Estas funciones ayudan a mantener la integridad de la estructura:

- **estaVacia():** Retorna verdadero si no hay nadie en la fila.
- **estaLlena():** Compara el tamaño actual (cola.size()) con la capacidad permitida.

- **tamanio():** Indica el número exacto de personas registradas actualmente.

Diferencia con la Pila (Stack)

Mientras que en el anterior la **Pila** el último en llegar era el primero en salir (como una pila de platos), en esta **Cola** el primero que agregas es el primero que el método quitar() procesará (como una fila en el banco).

```
Capacidad de la cola: 1

--- MENU ---
1. Agregar persona
2. Quitar persona
3. Mostrar contenido
4. Verificar si esta vacia
5. Verificar si esta llena
6. Mostrar tama±o
7. Salir
Opcion: 1
Nombre: Karla
Edad: 18
Persona agregada.

--- MENU ---
1. Agregar persona
2. Quitar persona
3. Mostrar contenido
4. Verificar si esta vacia
5. Verificar si esta llena
6. Mostrar tama±o
7. Salir
Opcion: 3
Contenido:
Karla - 18
```

Que es lo que imprime y explicación:

1. **Definición de Capacidad:**
   - Se establece la **Capacidad de la cola: 1**. Esto limita la estructura a contener un solo objeto Persona a la vez.
2. **Inserción de Datos (Opción 1):**
   - Seleccionas la **Opcion: 1** para agregar una persona.
   - El programa solicita y recibe los datos: **Nombre: Karla** y **Edad: 18**.
   - Se crea el objeto y se guarda en la cola, imprimiendo: **"Persona agregada."**.
3. **Consulta de Contenido (Opción 3):**
   - Seleccionas la **Opcion: 3** para ver quién está en la fila.
   - El programa imprime el encabezado **"Contenido:"** seguido de los datos del objeto almacenado: **Karla - 18**.

Errores y Soluciones: no hubo errores.

LISTA (manera dinámica por medio de las librerías o clases proporcionadas por el IDE)

Dato DB

main.cpp Estructura.h Lista.h

```cpp
#ifndef LISTA_H
#define LISTA_H

#include <iostream>
#include <vector>
#include "Estructura.h"

using namespace std;

class Lista : public Estructura {
private:
    vector<int> lista;
    int capacidad;

public:
    Lista(int cap) {
        capacidad = cap;
    }

    void agregar() {
        if (!estaLlena()) {
            int valor;
            cout << "Ingresa un numero: ";
            cin >> valor;

            lista.push_back(valor);
            cout << "Elemento agregado.\n";
        } else {
            cout << "La lista esta llena.\n";
        }
    }

    void quitar() {
        if (!estaVacia()) {
            int pos;
            cout << "Posicion a eliminar (0 a " << lista.size()-1 << "): ";
            cin >> pos;

            if (pos >= 0 && pos < lista.size()) {
                cout << "Eliminado: " << lista[pos] << endl;
                lista.erase(lista.begin() + pos);
            } else {
                cout << "Posicion invalida.\n";
            }
        } else {
            cout << "La lista esta vacia.\n";
        }
    }

    void mostrar() {
        if (estaVacia()) {
            cout << "La lista esta vacia.\n";
            return;
        }

        cout << "Contenido: ";
        for (int i = 0; i < lista.size(); i++) {
            cout << lista[i] << " ";
        }
        cout << endl;
    }

    bool estaVacia() {
        return lista.empty();
    }

    bool estaLlena() {
        return lista.size() == capacidad;
    }

    int tamanio() {
        return lista.size();
    }
};

#endif
```

Objetivo del Código

El objetivo principal es implementar una **Lista Secuencial** con control de capacidad. A diferencia de las Pilas (donde solo sacas el último) o las Colas (donde solo sacas el primero), esta estructura permite **eliminar elementos en cualquier posición específica** que elija el usuario, ofreciendo mayor flexibilidad para organizar datos.

Explicación del Código

1. Estructura y Almacenamiento

- **vector<int> lista**: Utiliza un contenedor de la biblioteca estándar para guardar los números de forma contigua en memoria.
- **int capacidad**: Define el límite máximo de elementos que la lista puede contener, el cual se asigna mediante el **constructor** al crear el objeto.

2. Método agregar()

- Primero verifica si hay espacio disponible mediante !estaLlena().

- Si hay espacio, solicita un número al usuario y lo añade al final de la lista usando push_back(valor).
- Si no, avisa que la lista ha alcanzado su límite.

3. Método quitar()

Esta es la parte más flexible del código:

- Verifica que la lista no esté vacía.
- Solicita una **posición (índice)** para eliminar.
- **Validación Crítica**: Revisa que la posición ingresada esté dentro del rango válido (desde 0 hasta el tamaño actual - 1).
- **lista.erase(...)**: Elimina el elemento en esa posición exacta y desplaza los elementos restantes para cerrar el hueco.

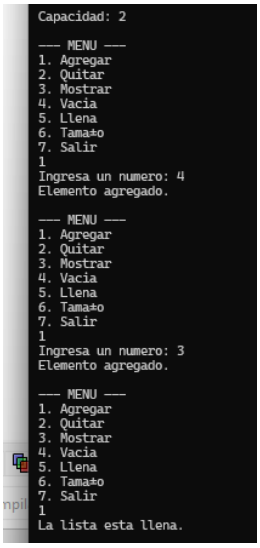
4. Método mostrar()

- Recorre la lista desde el índice 0 hasta el final usando un ciclo for.
- Imprime cada número separado por un espacio para que el usuario pueda ver el estado actual de la estructura.

5. Funciones de Control

- **estaVacia()**: Devuelve true si el vector no tiene elementos.
- **estaLlena()**: Devuelve true si el número de elementos iguala la capacidad definida inicialmente.
- **tamanio()**: Informa cuántos elementos hay guardados actualmente.

Diferencia con Pila/Cola: En la Pila o Cola no eliges qué borrar (siempre es el extremo), aquí sí puedes elegir la "posición a eliminar"



```
Capacidad: 2

--- MENU ---
1. Agregar
2. Quitar
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
1
Ingresa un numero: 4
Elemento agregado.

--- MENU ---
1. Agregar
2. Quitar
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
1
Ingresa un numero: 3
Elemento agregado.

--- MENU ---
1. Agregar
2. Quitar
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
1
La lista esta llena.
```

Que es lo que imprime y explicación:

Primera y Segunda Inserción

- **Acción**: Seleccionas la **Opción 1** dos veces.
- **Entrada**: Ingresas el número **4** y luego el número **3**.
- **Lo que imprime**: "Elemento agregado." en ambas ocasiones.
- **Explicación**: El método agregar() de tu archivo Lista.h verifica si la lista está llena. Como el tamaño actual (0 y luego 1) es menor a la capacidad (2), el programa permite usar lista.push_back(valor) para guardar los números.

2. Intento de Tercera Inserción (Fallo)

- **Acción**: Intentas seleccionar la **Opción 1** por tercera vez.
- **Lo que imprime**: **"La lista esta llena."**
- **Explicación**: Aquí entra en juego la función estaLlena(). Esta función compara el tamaño del vector con la capacidad definida (lista.size() == capacidad). Como ya tienes dos elementos (el 4 y el 3), la condición es verdadera y el programa ejecuta el bloque else de tu método agregar(), bloqueando la entrada de más datos para proteger la memoria.

Errores y Soluciones: no hubo errores.

Nuevo tipo de dato

main.cpp Estructura.h Lista.h

```cpp
#ifndef LISTA_H
#define LISTA_H

#include <iostream>
#include <vector>
#include <string>
#include "Estructura.h"

using namespace std;

// ?? Tipo de dato propio
class Persona {
public:
    string nombre;
    int edad;

    Persona() {
        nombre = "";
        edad = 0;
    }

    Persona(string n, int e) {
        nombre = n;
        edad = e;
    }
};

class Lista : public Estructura {
private:
    vector<Persona> lista;
    int capacidad;

public:
    Lista(int cap) {
        capacidad = cap;
    }
```

```cpp

    void agregar() {
        if (!estaLlena()) {
            string nombre;
            int edad;

            cout << "Nombre: ";
            cin >> nombre;
            cout << "Edad: ";
            cin >> edad;

            lista.push_back(Persona(nombre, edad));
            cout << "Persona agregada.\n";
        } else {
            cout << "La lista esta llena.\n";
        }
    }

    void quitar() {
        if (!estaVacia()) {
            int pos;
            cout << "Posicion a eliminar (0 a " << lista.size()-1 << "): ";
            cin >> pos;

            if (pos >= 0 && pos < lista.size()) {
                cout << "Eliminando: "
                     << lista[pos].nombre
                     << " (" << lista[pos].edad << ")\n";

                lista.erase(lista.begin() + pos);
            } else {
                cout << "Posicion invalida.\n";
            }
        } else {
            cout << "La lista esta vacia.\n";
        }
```

```cpp
    }

    void mostrar() {
        if (estaVacia()) {
            cout << "La lista esta vacia.\n";
            return;
        }

        cout << "Contenido:\n";
        for (int i = 0; i < lista.size(); i++) {
            cout << lista[i].nombre
                 << " - " << lista[i].edad << endl;
        }
    }

    bool estaVacia() {
        return lista.empty();
    }

    bool estaLlena() {
        return lista.size() == capacidad;
    }

    int tamanio() {
        return lista.size();
    }
};

#endif
```

Objetivo del Código

El objetivo principal es crear una estructura de datos tipo **Lista** capaz de almacenar información compleja (nombres y edades) en lugar de simples números. A diferencia de una pila o una cola, esta lista permite al usuario **gestionar registros específicos**, dándole la libertad de eliminar a cualquier persona de la lista basándose en su posición, siempre respetando un límite de capacidad máxima.

Explicación del Código

1. Clase Persona

Define el tipo de objeto que guardaremos.

- **Atributos:** nombre (string) y edad (int).
- **Constructores:** Tiene uno por defecto que inicializa valores vacíos y otro parametrizado que permite crear una persona con datos específicos en una sola línea de código.

2. Atributos de la Clase Lista

- **vector<Persona> lista:** Utiliza un vector dinámico de C++, pero ahora está configurado para guardar objetos de la clase Persona.
- **capacidad:** Variable privada que define cuántas personas caben en la lista. Se asigna mediante el constructor Lista(int cap).

4. Método agregar()

Verifica si hay espacio con estaLlena().

- Si hay lugar, pide al usuario el nombre y la edad.
- **lista.push_back(Persona(nombre, edad))**: Aquí crea un objeto Persona temporal con los datos ingresados y lo añade al final del vector.

4. Método quitar()

Este es el método más flexible:

- Primero pide la **posición exacta** que se desea borrar (por ejemplo, el registro 0, 1, 2...).
- Valida que la posición exista dentro de los límites actuales del vector (pos >= 0 && pos < lista.size()).
- **lista.erase(lista.begin() + pos)**: Localiza el objeto en esa posición y lo elimina, recorriendo automáticamente a los demás para que no queden huecos en la memoria.

5. Método mostrar()

- Recorre el vector con un ciclo for.
- Accede a los atributos internos de cada objeto almacenado para imprimir: lista[i].nombre y lista[i].edad.

6. Métodos de Estado

- **estaVacia()**: Usa lista.empty() para saber si no hay registros.
- **estaLlena()**: Compara el tamaño actual con la capacidad límite.
- **tamanio()**: Devuelve la cantidad de personas registradas actualmente.

```
Capacidad: 1

--- MENU ---
1. Agregar
2. Quitar
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
1
Nombre: Pamela
Edad: 36
Persona agregada.

--- MENU ---
1. Agregar
2. Quitar
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
1
La lista esta llena.

--- MENU ---
1. Agregar
2. Quitar
3. Mostrar
4. Vacia
5. Llena
6. Tama±o
7. Salir
```

Que es lo que imprime y explicación:

Configuración de Capacidad: El programa inicia indicando una **Capacidad: 1**, lo que restringe la lista a un solo registro de tipo Persona.

Primera Inserción (Exitosa):

- Seleccionas la opción **1** (Agregar).
- Ingresas el nombre **Pamela** y la edad **36**.
- El programa imprime: **"Persona agregada."**.

Segunda Inserción (Fallida):

- Intentas seleccionar nuevamente la opción **1**.
- Al estar ya ocupado el único espacio disponible, el programa imprime: **"La lista esta llena."**.

Errores y Soluciones: no hubo errores.